



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-408Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Модуль интеграции колоночного файлового формата Vortex в открытый табличный формат и сравнение со стандартными файловыми форматами

Автор ВКРБ: Кочкожаров Иван Вячеславович ()

Руководитель: Гаврилов Евгений Сергеевич ()

Консультант: ()

Консультант: ()

Рецензент: ()

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ()

_____ 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 64 страниц, 4 рисунков, 12 таблиц, 30 использованных источников, 2 приложений.

БОЛЬШИЕ ДАННЫЕ, АРАСНЕ PAIMON, КОЛОНОЧНЫЙ ФОРМАТ, VORTEX, PARQUET, ORC, LAKEHOUSE, LSM-ДЕРЕВО, КОДИРОВАНИЕ ДАННЫХ, НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ, АРАСНЕ FLINK

Объект исследования — колоночные файловые форматы (Parquet, ORC, Vortex) в составе lakehouse-системы Apache Paimon; предмет — алгоритмы кодирования, сжатия и пропуска данных и их влияние на производительность. Цель — определить область применимости формата Vortex в Apache Paimon, разработать модуль его интеграции и провести сравнительный анализ.

Выполнен обзор lakehouse-архитектур и колоночных форматов; разработан и интегрирован модуль поддержки Vortex (FileFormat, конвертер предикатов, JNI); проведено функциональное тестирование в контейнерах; развёрнуто кластерное окружение средствами Ansible; проведены две серии нагрузочных экспериментов — batch-аналитика TPC-DS (100 GB) и near-real-time-сценарий с потоковой записью и параллельными запросами.

Результаты: на batch-аналитике Vortex быстрее Parquet на 82 из 103 запросах, среднее ускорение $\approx 25\%$, пиковое — до 80 % (основная область применимости). На NRT-сценарии Vortex эквивалентен Parquet и ORC по записи; по медиане чтения — быстрее формат в обеих фазах (9/13 запросов), особенно на запросах с селективным предикатом, диапазоным фильтром по строке-префиксу и в многошаговой аналитике; на простой агрегации без предиката незначительно уступает (отсутствие проталкивания агрегатов в интерфейсе Paimon). ORC даёт узкие хвосты задержки и наиболее компактный объём хранилища (на 5–18 % меньше Parquet и Vortex). Parquet ловит эпизодические выбросы времени запросов в моменты сброса по чекпойнту, что согласуется с архитектурным выигрышем Vortex за счёт независимого нативного пула соединений с хранилищем. Область применения — проектирование витрин данных на Apache Paimon, развитие его экосистемы.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1 АНАЛИТИЧЕСКИЙ ОБЗОР	10
1.1 Эволюция систем хранения аналитических данных	10
1.2 Открытые табличные форматы	11
1.3 Колоночные файловые форматы хранения данных	13
1.4 Архитектура Apache Paimon	16
1.5 Кодирование, сжатие и пропуск данных	19
1.6 Постановка задачи	22
2 РАЗРАБОТКА МОДУЛЯ ИНТЕГРАЦИИ И РАЗВЁРТЫВАНИЕ ОКРУЖЕНИЯ	25
2.1 Модуль интеграции формата Vortex в Apache Paimon	25
2.2 Функциональное тестирование в контейнерах	28
2.3 Развёртывание окружения нагрузочного тестирования	30
2.4 Методика экспериментов	31
2.4.1 Сценарий 1: пакетная аналитика на наборе TPC-DS	32
2.4.2 Сценарий 2: near-real-time-витрина при потоковой записи	32
2.5 Проблемы, выявленные и решённые в ходе работы	37
2.5.1 Сборка и локальное тестирование	37
2.5.2 Развёртывание кластера и автоматизация	37
2.5.3 Хранилище: каталог Apache Paimon поверх Apache Ozone	38
2.5.4 Настройка потокового сценария	38
2.5.5 Отклонённые гипотезы по тонкой настройке	39
3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ	41
3.1 Сценарий 1: пакетная аналитика на наборе TPC-DS	41
3.2 Сценарий 2: near-real-time-витрина при потоковой записи	44
3.3 Паттерны по типам запросов и архитектурные наблюдения	49
3.4 Рекомендации по применению форматов хранения	53
ЗАКЛЮЧЕНИЕ	55
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	58
ПРИЛОЖЕНИЕ А Исходные тексты проекта	61
ПРИЛОЖЕНИЕ Б Параметры экспериментов	63

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

кодирование колонки — преобразование последовательности однотипных значений столбца в более компактное двоичное представление с использованием статистических свойств данных (повторов, ограниченного диапазона, общих префиксов и т. п.)

пропуск данных — (англ. data skipping) приём оптимизации чтения, при котором по статистикам (минимум, максимум, число null-значений) блоков данных заранее определяется, что блок не содержит подходящих под предикат строк, и его чтение пропускается

слияние (компакция) — (англ. compaction) фоновая операция в LSM-дереве, объединяющая несколько отсортированных файлов одного уровня в меньшее число файлов следующего уровня с удалением перезаписанных и удалённых записей

файловый формат — спецификация двоичного представления табличных данных в файле, включающая разбиение на блоки, схему хранения значений (построчное или поколоночное), способы кодирования, сжатия и встроенные метаданные (статистики, индексы)

открытый табличный формат — (англ. open table format) спецификация метаданных поверх набора файлов данных, обеспечивающая представление их как единой таблицы с поддержкой транзакций, эволюции схемы, моментальных снимков и параллельного доступа; примеры — Apache Iceberg, Delta Lake, Apache Hudi, Apache Paimon

колоночный формат хранения — формат, в котором значения одного столбца таблицы располагаются в файле непрерывно; обеспечивает высокую степень сжатия однотипных данных и чтение только запрашиваемых столбцов

предикат — логическое условие фильтрации строк (например, `col > 100 AND col < 200`); может передаваться в файловый формат для отсечения блоков данных на уровне декодера (predicate pushdown)

слияние при чтении — (англ. merge-on-read) стратегия таблиц с первичным ключом, при которой обновления и удаления записываются как новые версии, а актуальное состояние строки вычисляется во время чтения объединением версий из всех уровней LSM-дерева по полю последовательности

lakehouse — архитектура хранения данных, объединяющая дешёвое масштабируемое хранилище объектов озера данных (data lake) с транзакционными гарантиями, управлением схемой и SQL-доступом, характерными для хранилищ данных (data warehouse)

LSM-дерево — (англ. Log-Structured Merge-tree) структура данных, оптимизированная под интенсивную запись: новые данные буферизуются в памяти и сбрасываются на диск отсортированными неизменяемыми файлами (уровень 0), которые затем фоновым слиянием перемещаются на нижележащие уровни

OLAP — (англ. Online Analytical Processing) класс нагрузок, характеризующихся сложными аналитическими запросами с агрегацией над большими объёмами данных и редкими изменениями; противопоставляется OLTP

OLTP — (англ. Online Transaction Processing) класс нагрузок, характеризующихся большим числом коротких транзакций чтения и записи отдельных записей по ключу

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

ALP — Adaptive Lossless floating-Point compression

API — Application Programming Interface

CBO — Cost-Based Optimizer

CDC — Change Data Capture

CPU — Central Processing Unit

DWH — Data Warehouse

FSST — Fast Static Symbol Table

GPU — Graphics Processing Unit

HDFS — Hadoop Distributed File System

JNI — Java Native Interface

JVM — Java Virtual Machine

LSM — Log-Structured Merge-tree

NRT — Near Real-Time

OLAP — Online Analytical Processing

OLTP — Online Transaction Processing

ORC — Optimized Row Columnar

RLE — Run-Length Encoding

S3 — Simple Storage Service

SIMD — Single Instruction, Multiple Data

SQL — Structured Query Language

TPC-DS — Transaction Processing Performance Council Decision Support benchmark

ВКР — выпускная квалификационная работа

ВВЕДЕНИЕ

Современные организации придерживаются стратегии накопления всех доступных данных в расчёте на их будущее использование в задачах, которые на момент сбора ещё не сформулированы. Эта практика породила потребность в дешёвом, масштабируемом и при этом аналитически пригодном хранилище. Классические хранилища данных (data warehouse) обеспечивают транзакционность, управление схемой и быстрый SQL-доступ, но плохо масштабируются по стоимости и не приспособлены к слабоструктурированным данным. Озёра данных (data lake) на основе распределённых файловых систем и объектных хранилищ дешёвы и масштабируемы, но не дают транзакционных гарантий и управления метаданными. Архитектура lakehouse [1] объединяет достоинства обоих подходов: поверх дешёвого объектного хранилища строится слой открытого табличного формата, обеспечивающий ACID-транзакции, эволюцию схемы, моментальные снимки и параллельный доступ. К числу таких систем относятся Apache Iceberg [2], Delta Lake [3], Apache Hudi [4] и Apache Paimon [5].

При проектировании lakehouse-хранилища одним из ключевых решений является выбор файлового формата, в котором физически хранятся данные. От него зависят степень сжатия (а значит, объём хранилища и стоимость), скорость записи, скорость аналитических запросов и эффективность пропуска нерелевантных данных. Де-факто стандартом колоночного хранения в экосистеме больших данных является Apache Parquet; широко применяется также Apache ORC. Однако оба формата разрабатывались более десяти лет назад, их форматы кодирования отражают возможности процессоров того времени и не используют современные приёмы — векторизованное декодирование с явной SIMD-параллельностью, адаптивное сжатие чисел с плавающей точкой, кодирование строк по схожести префиксов. Эти приёмы реализованы в новом колоночном формате Vortex [6; 7], разрабатываемом с прицелом на нативное (написанное на Rust) векторизованное выполнение и переносимость метаданных. Вопрос о том, в каких сценариях преимущества Vortex проявляются на практике и можно ли интегрировать его в существующую lakehouse-систему, на момент начала работы оставался открытым.

Актуальность работы определяется, во-первых, практической потребностью в обоснованном выборе формата хранения при построении витрин данных на Apache Paimon, а во-вторых, тем, что Vortex — активно развивающийся формат, поддержка которого в Apache Paimon отсутствовала, и интеграция вместе с экспериментальной оценкой представляет самостоятельный интерес как для сообщества Apache Paimon, так и для организации, в инфраструктуре которой выполнялась работа.

Цель работы — определить область применимости колоночного формата Vortex в lakehouse-системе Apache Paimon, разработать модуль его интеграции и выполнить сравнительный анализ форматов хранения (Parquet, ORC, Vortex) на представительных нагрузках.

Для достижения цели поставлены следующие **задачи**:

- а) провести анализ подходов к хранению аналитических данных — от хранилищ данных (Hive Metastore, Impala) к архитектуре lakehouse и открытым табличным форматам;
- б) провести обзор колоночных файловых форматов (Parquet, ORC, Vortex, Lance) и обосновать исключение строчного формата Avro из сравнения;
- в) изучить архитектуру Apache Paimon — LSM-дерево, таблицы с первичным ключом, слияние при чтении, — и алгоритмы кодирования, сжатия и пропуска данных, применяемые форматами;
- г) разработать модуль интеграции файлового формата Vortex в Apache Paimon;
- д) провести функциональное тестирование полученного артефакта локально в контейнерах;
- е) развернуть на кластере окружение нагрузочного тестирования средствами Ansible;
- ж) провести серию экспериментов и собрать метрики производительности;
- и) сформулировать рекомендации по применению форматов (опционально).

Объект исследования — колоночные файловые форматы хранения данных в составе lakehouse-системы Apache Paimon. **Предмет исследования** — алгоритмы кодирования, сжатия и пропуска данных, реализуемые этими форматами, и их влияние на производительность чтения и записи таблиц.

Методы исследования: анализ исходного кода и документации Apache Paimon, Apache Flink и формата Vortex; разработка программного модуля на языках Java и Rust; функциональное тестирование в контейнеризованном окружении; нагрузочное тестирование на кластере с использованием стандартизированного набора запросов TPC-DS и специально разработанного потокового сценария; статистическая обработка результатов измерений (усреднение по повторным запускам, оценка разброса).

Практическая значимость. Разработанный модуль интеграции расширяет перечень поддерживаемых Apache Paimon форматов хранения. Полученные количественные результаты и рекомендации применимы при проектировании витрин данных и настройке lakehouse-хранилищ. Выявленное архитектурное ограничение интерфейса форматов Apache Paimon (отсутствие проталкивания агрегатов) указывает направление дальнейшего развития системы.

Отчёт состоит из введения, трёх разделов, заключения, списка использованных источников и приложений. Первый раздел содержит аналитический обзор: эволюцию систем хранения, открытые табличные форматы, колоночные файловые форматы, архитектуру Apache Paimon, алгоритмы кодирования и пропуска данных, а также постановку задачи. Второй раздел посвящён разработке модуля интеграции формата Vortex, функциональному тестированию, развёртыванию окружения нагрузочного тестирования, методике экспериментов и обнаруженным в ходе работы проблемам. Третий раздел содержит результаты экспериментов, их анализ по типам запросов и рекомендации по применению форматов.

1 АНАЛИТИЧЕСКИЙ ОБЗОР

В разделе рассматриваются эволюция систем хранения аналитических данных, открытые табличные форматы как ключевой компонент архитектуры lakehouse, колоночные файловые форматы хранения, архитектура системы Apache Paimon и применяемые форматами алгоритмы кодирования, сжатия и пропуска данных. По итогам обзора формулируется постановка задачи.

1.1 Эволюция систем хранения аналитических данных

Исторически аналитическая обработка данных опиралась на хранилища данных [8; 9] (data warehouse) — специализированные реляционные системы, в которые данные загружаются по расписанию через процессы ETL, приводятся к единой схеме «звезда» или «снежинка» и затем обслуживают запросы бизнес-аналитики. Хранилище данных обеспечивает строгую типизацию, управление схемой, транзакционность и предсказуемую производительность запросов. Платой за это являются высокая стоимость хранения (вычислительные узлы и хранилище связаны и масштабируются совместно), жёсткость схемы (изменение требует перепроектирования ETL) и неспособность работать со слабоструктурированными данными — журналами, событиями, телеметрией.

С появлением распределённой файловой системы Hadoop (HDFS) сформировалась альтернативная архитектура — озеро данных (data lake). Данные складываются в HDFS или объектное хранилище «как есть», в исходных форматах, а схема применяется во время чтения (schema-on-read). Слой совместимости с SQL обеспечивался Hive Metastore [10] — сервисом каталога, хранящим описания таблиц (расположение файлов, схему, разбиение на разделы) в реляционной базе, — и движками SQL поверх него: Apache Hive [10], затем Apache Impala [11], Presto, Spark SQL. Озеро данных дёшево и масштабируемо, позволяет хранить любые данные, но не даёт транзакционных гарантий: одновременная запись и чтение, частичные обновления, атомарная замена набора файлов не поддерживаются на уровне файловой системы. Hive-таблица, по сути, — это директория с файлами; согласованность при изменениях обеспечивается только дисциплиной на стороне приложений.

Ограничения обоих подходов привели к появлению архитектуры

lakehouse. Идея состоит в том, чтобы сохранить дешёвое масштабируемое объектное хранилище озера данных, но добавить поверх него слой метаданных, который превращает набор файлов в полноценную таблицу с ACID-транзакциями, эволюцией схемы, моментальными снимками (snapshot isolation), управлением версиями и параллельным доступом. Этот слой и называется *открытым табличным форматом*. Вычислительные движки (Spark, Flink, Trino и другие) работают с таблицей через спецификацию табличного формата, а не напрямую с файлами; разделение хранения и вычислений сохраняется, поскольку и данные, и метаданные лежат в объектном хранилище. Современные реализации — Apache Iceberg [2], Delta Lake [3], Apache Hudi [4] и Apache Paimon [5] — различаются деталями устройства метаданных и набором поддерживаемых сценариев, но разделяют общую модель.

Особое место среди них занимает Apache Paimon. В отличие от Iceberg и Delta Lake, ориентированных в первую очередь на таблицы только для добавления (append-only) и пакетную загрузку, Paimon изначально проектировался под потоковую обработку и таблицы с первичным ключом: в его основе лежит LSM-дерево, что делает эффективными сценарии захвата изменений (Change Data Capture) и построения near-real-time-витрин. Именно поэтому Apache Paimon выбран целевой системой настоящей работы.

1.2 Открытые табличные форматы

Открытый табличный формат определяет, как из набора файлов данных, лежащих в объектном хранилище, собирается логическая таблица. Минимальный набор обязанностей табличного формата включает: ведение списка файлов, составляющих текущее состояние таблицы; атомарную фиксацию изменений (добавление и удаление файлов одной транзакцией); изоляцию снимков (читатель видит согласованное состояние на момент начала чтения, не затрагиваемое параллельной записью); хранение схемы и её эволюцию; статистики по файлам для пропуска нерелевантных данных; разбиение на разделы. Рассмотрим основные реализации.

Apache Iceberg [2] ведёт метаданные в виде дерева: файл метаданных таблицы ссылается на список манифестов (manifest list), каждый манифест перечисляет файлы данных с их статистиками. Фиксация транзакции — это атомарная замена указателя на новый файл метаданных. Iceberg поддерживает

скрытое разбиение на разделы (partition evolution), эволюцию схемы без перезаписи данных, ветвление и теги снимков. Удаления реализуются через позиционные и равенственные delete-файлы. Iceberg — наиболее широко принятый в индустрии формат, поддержанный всеми крупными движками.

Delta Lake [3] использует журнал транзакций — упорядоченную последовательность JSON-файлов, каждый из которых описывает добавленные и удалённые файлы данных; периодически журнал уплотняется в контрольную точку в формате Parquet. Delta поддерживает обновления и удаления через перезапись затронутых файлов (copy-on-write) либо через deletion vectors. Тесно интегрирован с Apache Spark.

Apache Hudi [4] ориентирован на инкрементальную обработку и upsert по ключу записи. Поддерживает два типа таблиц: copy-on-write (обновления порождают перезапись файлов) и merge-on-read (обновления пишутся в журнальные файлы, состояние вычисляется при чтении). Ведёт временную шкалу (timeline) операций и индекс, сопоставляющий ключи записей файлам.

Apache Paimon [5] (изначально Flink Table Store) выделяется тем, что состояние таблицы с первичным ключом организовано как LSM-дерево, размещённое в объектном хранилище. Свежие изменения сбрасываются в файлы уровня 0, фоновое слияние перемещает их на нижележащие уровни; при чтении актуальная версия строки получается объединением версий из всех уровней по полю последовательности. Метаданные ведутся аналогично Iceberg: снимок ссылается на манифесты, манифесты — на файлы данных со статистиками. Такое устройство делает Paimon эффективным для потоковой записи, захвата изменений и near-real-time-витрин — сценариев, в которых таблица постоянно обновляется небольшими порциями, а не перезаписывается целиком. Apache Paimon поддерживает несколько файловых форматов хранения данных, выбираемых параметром таблицы: Apache Parquet (по умолчанию), Apache ORC и Apache Avro; добавление поддержки формата Vortex составляет практическую часть настоящей работы.

Для задач настоящей работы важно, что табличный формат и файловый формат — разные уровни абстракции: первый управляет составом и версиями набора файлов, второй — двоичным представлением данных внутри одного файла. Производительность чтения и записи определяется обоими уровнями; в работе фиксируется табличный формат (Apache Paimon) и варьируется файловый формат.

1.3 Колоночные файловые форматы хранения данных

Файловый формат задаёт, как табличные данные раскладываются в байты внутри файла. Принципиальное различие — построчное (row-oriented) и поколоночное (columnar) хранение. При построчном хранении значения одной строки лежат рядом, что выгодно для точечных операций по ключу (прочитать или записать целую запись) и для потоковой записи (строка добавляется в конец без переупорядочивания). При поколоночном хранении рядом лежат значения одного столбца: это даёт высокую степень сжатия (однотипные данные сжимаются лучше), позволяет читать только запрашиваемые столбцы (column pruning) и эффективно применять векторизованные операции к плотным массивам значений. Аналитические нагрузки (OLAP) — сканирование, фильтрация и агрегация по немногим столбцам широких таблиц — естественно ложатся на колоночное хранение, поэтому далее рассматриваются именно колоночные форматы [12; 13].

Apache Parquet [14] — де-факто стандарт колоночного хранения в экосистеме больших данных. Файл делится на группы строк (row groups, типично десятки или сотни мегабайт); внутри группы строк каждый столбец хранится отдельным фрагментом (column chunk), который, в свою очередь, разбивается на страницы (pages, типично около 1 MB). Страница — единица сжатия и кодирования. Parquet применяет словарное кодирование (dictionary encoding) для столбцов с небольшим числом различных значений, кодирование длин серий (run-length encoding) и упаковку в минимально необходимое число бит (bit-packing) для целочисленных и словарных индексов, дельта-кодирование для возрастающих последовательностей, а поверх — блочное сжатие (Snappy, ZSTD, GZIP). Для пропуска данных Parquet хранит статистики (минимум, максимум, число null) на уровне группы строк и страницы, а также опционально — фильтры Блума [15]. Чтение в JVM-окружении выполняется библиотекой `parquet-mr`; в Apache Paimon используется её затенённая (shaded) сборка.

Apache ORC [16] (Optimized Row Columnar) близок по идеологии к Parquet, но исторически развивался в экосистеме Apache Hive. Файл делится на полосы (stripes, типично около 64 MB); каждая полоса содержит данные столбцов, индексные данные и нижний колонтитул. ORC применяет «лёгкое» кодирование (RLE для целых, словарное для строк) и «тяжёлое» блочное

сжатие (ZLIB, Snappy, ZSTD); ведёт встроенные индексы со статистиками на уровне полосы и группы строк (по 10 000 строк), что делает его особенно эффективным на точечных и узкодиапазонных запросах. Реализация — нативная для JVM библиотека `orc-core`.

Vortex [6; 7] — современный колоночный формат, разрабатываемый на языке Rust с прицелом на нативное векторизованное выполнение. Ключевые особенности. Во-первых, система кодирования: целочисленные данные кодируются по схеме FastLanes [17] — бит-упаковка, ориентированная на SIMD-декодирование (значения раскладываются так, чтобы распаковка выполнялась векторными инструкциями процессора без ветвлений); числа с плавающей точкой — кодеком ALP [18] (Adaptive Lossless floating-Point), который без потерь представляет вещественные значения как целые числа с общим масштабом, а исключения хранит отдельно; строки — кодеком FSST [19] (Fast Static Symbol Table), строящим словарь коротких подстрок (символов) и заменяющим повторяющиеся фрагменты ссылками на словарь, что особенно эффективно на строках с общими префиксами (URL, идентификаторы, пути). Кодеки в Vortex — расширяемые и каскадируемые: к закодированному массиву можно применить ещё один кодек. Во-вторых, физическая модель данных Vortex совместима с Apache Arrow [20], что позволяет передавать декодированные пакеты в движок без копирования через интерфейс Arrow C Data Interface. В-третьих, Vortex поддерживает проталкивание предикатов (predicate pushdown) на уровне нативного декодера: условие фильтрации преобразуется в нативное выражение, и блоки данных, заведомо не содержащие подходящих строк, не декодируются. В-четвёртых, метаданные файла (схема, статистики, расположение блоков) выделены в отдельный «футер», переносимый и читаемый независимо от данных. Доступ к нативной библиотеке Vortex из JVM осуществляется через Java Native Interface (JNI).

Lance [21] — колоночный формат, также реализованный на Rust, но ориентированный на нагрузки машинного обучения: быстрый произвольный доступ к строкам (random access), хранение векторных представлений (embeddings) и встроенные векторные индексы. Lance оптимизирует чтение отдельных строк и небольших выборок, что важно при обучении моделей и поиске ближайших соседей, но менее существенно для классической OLAP-аналитики со сканированием. Поддержка Lance в Apache Paimon на

момент работы существует, однако требует доработки для корректного слияния версий в таблицах с первичным ключом; по этой причине, а также ввиду иной целевой нагрузки, Lance в основной серии экспериментов настоящей работы не участвует (предварительные запуски показали отставание от остальных форматов в 2–3 раза на OLAP-сценарии).

Apache Avro [22] в сравнение не включён принципиально: это *строчный* формат. Avro хранит записи последовательно, со схемой в заголовке файла; он удобен для потоковой передачи и журналирования (дешев на запись, поддерживает эволюцию схемы), но не даёт ни поколонного сжатия, ни чтения отдельных столбцов, ни статистик для пропуска данных. Сравнивать его с колоночными форматами на аналитической нагрузке некорректно. Avro упоминается в работе лишь в связи с гипотезой о его использовании для самых свежих, ещё не уплотнённых файлов LSM-дерева; эта гипотеза рассмотрена и отклонена в практической части.

Сводная характеристика рассматриваемых форматов приведена в таблице 1.

Таблица 1 – Сравнительная характеристика колоночных файловых форматов

Формат	Реализация чтения	Отличительные черты
Parquet	JVM (<code>parquet-mr</code> , затенённая сборка в Paimon)	группы строк и страницы; словарное / RLE / bit-packing кодирование; блочное сжатие; статистики и фильтры Блума; зрелый универсальный формат
ORC	JVM (<code>orc-core</code>)	полосы (stripes); лёгкое и тяжёлое сжатие; встроенные индексы по 10 000 строк; преимущество на точечных и узкодиапазонных запросах
Vortex	нативная (Rust) через JNI	FastLanes (SIMD-bit-packing), ALP (числа с плавающей точкой), FSST (строки по схожести префиксов); каскадируемые кодеки; совместимость с Arrow; нативное проталкивание предикатов; переносимый футер метаданных
Lance	нативная (Rust) через JNI	быстрый произвольный доступ к строкам; векторные индексы; ориентация на нагрузки машинного обучения
Avro	JVM	строчный формат; дешёв на запись; нет поколоночного сжатия, column pruning и статистик; в сравнение не включён

1.4 Архитектура Apache Paimon

Apache Paimon [5] — открытый табличный формат, объединяющий слой метаданных в стиле Apache Iceberg с организацией данных в виде

LSM-дерева [23] в объектном хранилище. Ниже рассматриваются модель данных, устройство таблицы с первичным ключом, путь чтения и точка подключения файлового формата.

Модель данных и метаданные. Таблица Paimon — это директория в объектном хранилище, содержащая поддиректории данных и метаданных. Каждая фиксация (commit) порождает новый *снимок* (snapshot) — небольшой файл, ссылающийся на список *манифестов*. Манифест перечисляет файлы данных, добавленные или удалённые данной фиксацией, вместе со статистиками по столбцам каждого файла (минимум, максимум, число null). Чтение всегда выполняется относительно конкретного снимка, что обеспечивает изоляцию: параллельная запись создаёт новые снимки, не затрагивая тот, который читает потребитель. Старые снимки и недостижимые файлы удаляются по политике хранения (retention).

Разбиение и бакеты. Таблица может быть разбита на разделы (partitions) по значениям выбранных столбцов; внутри раздела данные распределяются по фиксированному числу *бакетов* (buckets) хешированием ключа. Бакет — единица параллелизма записи и единица LSM-дерева: каждый бакет имеет собственное LSM-дерево. Запись в бакет однопоточна, что устраняет конкуренцию за одно дерево; число бакетов задаёт максимальный параллелизм записи.

Таблица с первичным ключом и LSM-дерево. В таблице с первичным ключом строки идентифицируются ключом; обновления и удаления не перезаписывают данные на месте, а добавляются как новые версии. Внутри бакета новые записи буферизуются в памяти в отсортированном по ключу виде и при фиксации (по чекпойнту вычислительного движка) сбрасываются на диск одним отсортированным неизменяемым файлом — это *уровень 0* (L_0) LSM-дерева. Файлов L_0 накапливается по одному на чекпойнт. Фоновая операция *слияния* (compaction) объединяет несколько отсортированных серий (sorted runs) в файлы нижележащих уровней ($L_1, L_2, \dots$), отбрасывая перезаписанные и помеченные на удаление записи и укрупняя файлы; слияние не блокирует запись. Параметры num-sorted-run.compaction-trigger (число серий, при котором запускается слияние) и num-sorted-run.stop-trigger (порог обратного давления на запись) управляют его агрессивностью; target-file-size задаёт целевой размер укрупнённых файлов.

Схематически устройство показано на рисунке 1.

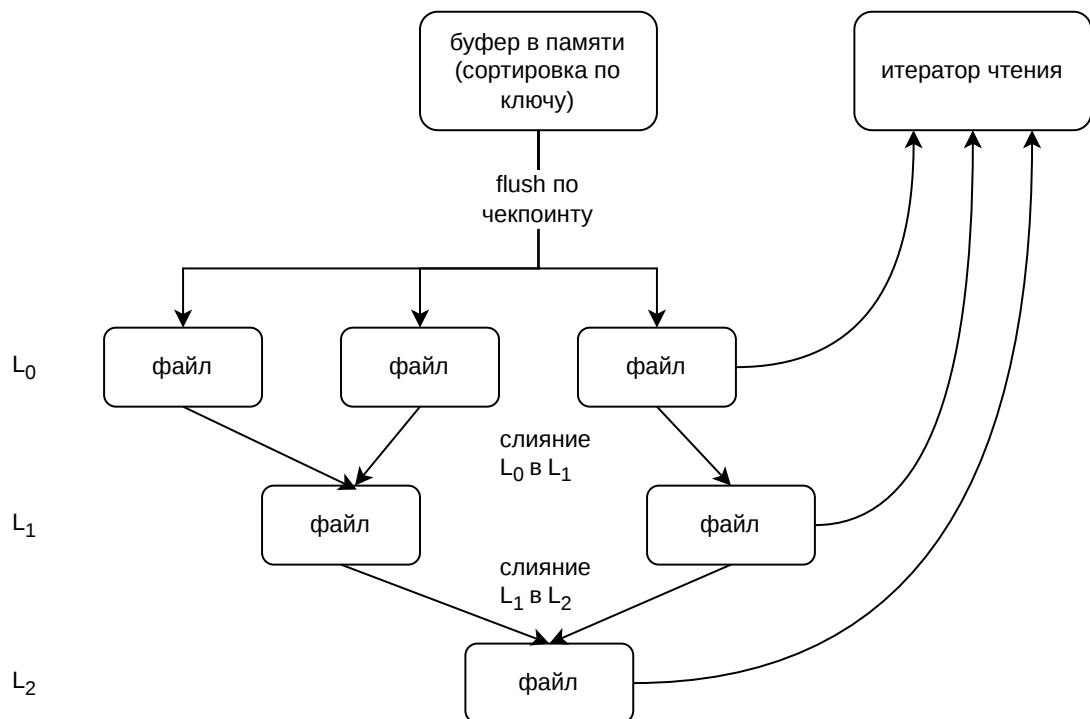


Рисунок 1 – Устройство LSM-дерева бакета в Apache Rafton: буферизация в памяти, сброс в L_0 по чекпойнту, фоновое слияние на нижележащие уровни, объединение версий итератором слияния при чтении

Поле последовательности и вид изменения. Чтобы итератор слияния выбирал актуальную версию строки, таблица настраивается на поле последовательности (`sequence.field`) — монотонно возрастающую величину (например, момент изменения), по максимуму которой определяется последняя версия ключа; поле вида изменения (`rowkind.field`) различает вставку, обновление и удаление. Эта схема естественно соответствует потокам захвата изменений (CDC).

Путь чтения. Чтение бакета порождает итератор слияния (`merge iterator`), который параллельно читает отсортированные серии всех уровней и выдаёт строки в порядке ключа, оставляя для каждого ключа версию с максимальным значением поля последовательности. Если файлов L_0 накопилось много (слияние не успело за записью), итератор вынужден открыть и слить больше файлов — стоимость чтения растёт. Перед открытием файла применяется пропуск данных по статистикам из манифеста: если

диапазоны значений столбцов файла не пересекаются с предикатом запроса, файл пропускается целиком; внутри файла дополнительно работает пропуск блоков средствами самого файлового формата. Декодированные данные передаются вычислительному движку пакетами строк (vectorized batches).

Точка подключения файлового формата. Файловый формат подключается к Paimon через интерфейс `FileFormat` (модуль `paimon-common`), который предоставляет фабрику записи (`FormatWriterFactory`) и фабрику чтения (`FormatReaderFactory`). Фабрика записи создаёт по пути в хранилище объект `FormatWriter`, принимающий строки и сериализующий их в файл выбранного формата. Фабрика чтения создаёт по пути, проекции столбцов и (опционально) предикату объект `FormatReaderFactory.FormatReader`, выдающий итератор пакетов строк. Реализация формата также объявляет, какие предикаты она способна принять для проталкивания, и формирует объект статистик по записанному файлу для манифеста. Существующие реализации — `paimon-format` (Parquet, ORC, Avro); добавление модуля `paimon-vortex`, реализующего этот интерфейс поверх нативной библиотеки Vortex, и составляет практическую часть работы.

Интеграция с вычислительным движком. В работе используется Apache Flink [24]: потоковая запись — через приёмник Paimon, чтение — через источник Paimon в пакетном режиме. Запись управляется чекпойнтами Flink: каждый чекпойнт фиксирует снимок Paimon. Интервал чекпойнтов задаёт частоту фиксаций: чем он больше, тем крупнее (и реже) файлы L_0 , но тем больше данных буферизуется и тем заметнее всплеск ввода-вывода при сбросе. Этот параметр существенен для near-real-time-сценария и отдельно варьируется в экспериментах.

1.5 Кодирование, сжатие и пропуск данных

Производительность колоночного формата определяется тремя группами механизмов: кодированием значений (компактное двоичное представление, использующее структуру данных), пропуском нерелевантных данных (отказ от чтения и декодирования блоков, заведомо не нужных запросу) и векторизованным декодированием (распаковка плотных массивов значений с использованием параллелизма процессора). Рассмотрим их.

Базовые кодеки. *Словарное кодирование* (dictionary encoding): для

столбца с небольшим числом различных значений строится словарь, а сами значения заменяются индексами в нём; выгодно для категориальных строк (страна, тип устройства, метод запроса). *Кодирование длин серий* (run-length encoding, RLE): последовательность одинаковых подряд идущих значений заменяется парой «значение, длина серии»; выгодно для отсортированных или малоизменчивых столбцов. *Упаковка в биты* (bit-packing): целые числа из ограниченного диапазона хранятся в минимально необходимом числе бит, а не в полном машинном слове. *Кодирование относительно опорного значения* (frame of reference, FOR): из всех значений вычитается минимум блока, остатки бит-упаковываются [25; 26]. *Дельта-кодирование*: хранятся разности соседних значений; выгодно для монотонных последовательностей (идентификаторы, временные метки). Поверх кодеков применяется блочное сжатие общего назначения (Snappy, ZSTD, LZ4). Parquet и ORC используют этот арсенал; различие в деталях (размеры блоков, выбор кодека, наличие фильтров Блума).

FastLanes. Vortex применяет для целочисленных данных схему бит-упаковки FastLanes [17], спроектированную под SIMD-декодирование. Обычная бит-упаковка требует при распаковке битовых сдвигов и масок с зависимостями по данным, что плохо ложится на векторные инструкции. FastLanes переставляет биты значений в памяти («транспонирует» их по полосам фиксированной ширины) так, чтобы распаковка k значений выполнялась небольшим числом векторных операций без ветвлений и без зависимостей между полосами. В сочетании с FOR и дельта-кодированием это даёт высокую пропускную способность декодирования целочисленных столбцов.

ALP. Для чисел с плавающей точкой Vortex применяет кодек ALP [18] (Adaptive Lossless floating-Point). Идея: многие вещественные значения в реальных данных — это десятичные дроби с небольшим числом знаков (координаты, цены, измерения), которые при умножении на подходящую степень десяти становятся целыми. ALP подбирает по выборке блока показатели степени, представляет основную массу значений как целые числа (далее — FOR и бит-упаковка), а немногочисленные значения, не укладывающиеся в эту модель (исключения), хранит отдельно в исходном виде. Преобразование обратимо без потерь. На столбцах с «настоящими» произвольными double ALP вырождается, но на типичных прикладных

данных существенно выигрывает у побайтового хранения.

FSST. Для строк Vortex применяет кодек FSST [19] (Fast Static Symbol Table). По выборке строит таблицу из не более чем 255 коротких подстрок («символов», длиной до 8 байт), часто встречающихся в данных, и кодирует строки последовательностью однобайтовых ссылок на символы (байт 0xFF экранирует буквальный байт). Декодирование — простая подстановка по таблице. FSST особенно эффективен на строках с регулярной структурой и общими префиксами: URL и пути (/api/v2/cat_017), идентификаторы (sess_0000001234, req_0000000042), доменные имена, — то есть на данных, которые в аналитических таблицах встречаются повсеместно. Существенно: эффективность FSST (и прочих кодеков) зависит от *порядка* строк внутри блока. В отсортированном блоке соседние строки похожи, символы переиспользуются плотно; в перемешанном блоке выигрыш меньше. Это объясняет, почему в LSM-дереве выгоды Vortex полнее проявляются на уплотнённых (отсортированных по ключу) данных, чем на свежих файлах L_0 .

Каскадирование кодеков. В Vortex кодеки — композируемые: к уже закодированному массиву применяется следующий кодек (например, словарь, затем бит-упаковка индексов словаря, затем FOR). Выбор каскада адаптивен — по выборке оценивается, какая комбинация даёт наилучшее сжатие при приемлемой стоимости декодирования.

Пропуск данных по статистикам. Файловый формат хранит для каждого блока (группа строк / страница в Parquet, полоса / группа в ORC, блок в Vortex) статистики — минимум, максимум, число null. Если предикат запроса (`col BETWEEN a AND b`, `col = v`, `col IS NOT NULL`) несовместим с диапазоном блока, блок не читается. Та же информация на уровне всего файла дублируется в манифесте Paimon, что позволяет отсечь файл ещё до его открытия. Тонкость: статистики по строковым столбцам часто хранятся усечёнными (truncate, например, до 16 байт первого несовпадающего символа) ради компактности; для столбцов, у которых различающая часть выходит за границу усечения, пропуск перестаёт работать. В работе используется режим полных статистик (`metadata.stats-mode = full`), чтобы исключить этот эффект из сравнения.

Проталкивание предикатов. Помимо пропуска целых блоков, формат может *проталкивать предикат* в декодер: вычислять условие на закодированных или частично декодированных данных и выдавать только

подходящие строки, не материализуя остальные. Vortex поддерживает проталкивание предикатов на уровне нативной библиотеки: Paimon-предикат преобразуется конвертером в нативное выражение Vortex (операции сравнения $=$, $\neq$, $<$, $\leq$, $>$, $\geq$, проверка на null, конъюнкция и дизъюнкция для всех поддерживаемых типов), и фильтрация выполняется в Rust-коде до пересечения границы JNI. Это даёт выигрыш на запросах с селективным предикатом: через границу JVM/native передаётся лишь малая доля строк.

Отсутствие проталкивания агрегатов. Принципиальное ограничение: интерфейс файловых форматов Apache Paimon (FileFormat) не предусматривает проталкивание агрегатов — операции SUM, COUNT, COUNT DISTINCT, GROUP BY выполняются вычислительным движком (Flink) над уже декодированными данными в JVM. Это архитектурное ограничение самого Paimon, а не конкретного формата. Для Parquet, у которого весь путь — от декодера до движка агрегации — находится в JVM, граница JNI отсутствует. Для Vortex данные сначала пересекают границу JNI (хотя и без копирования, через Arrow C Data Interface), а затем агрегируются в JVM; на запросах вида «полное сканирование + агрегация без селективного предиката» это даёт Vortex дополнительные накладные расходы. Устранение ограничения — проталкивание агрегатов в нативный SIMD-слой Vortex — лежит за рамками работы и обозначено как направление развития Apache Paimon.

Векторизованное декодирование. И Parquet, и ORC, и Vortex выдают данные движку не построчно, а пакетами (vectorized batches) — плотными массивами значений столбцов, к которым движок применяет векторизованные операции. Vortex дополнительно строит декодированные пакеты в раскладке Apache Arrow [27], что устраняет промежуточное копирование при передаче в Arrow-совместимые компоненты. Однако в Paimon результат всё равно адаптируется к внутреннему представлению строк Flink, поэтому полностью исключить накладные расходы границы не удаётся.

1.6 Постановка задачи

Проведённый обзор позволяет уточнить задачу работы. Apache Paimon — lakehouse-система с LSM-деревом, поддерживающая несколько файловых форматов; среди современных колоночных форматов выделяется Vortex с нативным векторизованным декодированием, специализированными

кодеками (FastLanes, ALP, FSST) и проталкиванием предикатов, но его поддержка в Apache Paimon отсутствует. Требуется определить, в каких сценариях Vortex даёт выигрыш, реализовать его интеграцию и измерить характеристики.

Конкретизация задач.

- а) **Разработка модуля интеграции.** Реализовать модуль `paimon-vortex`: фабрики записи и чтения поверх нативной библиотеки Vortex (доступ через JNI), конвертер Paimon-предикатов в нативные выражения Vortex, формирование статистик записанных файлов для манифеста, поддержку проекции столбцов и многопоточного чтения.
- б) **Функциональное тестирование.** Проверить корректность артефакта локально в контейнерах: запись и чтение таблиц с первичным ключом и таблиц только для добавления, слияние версий, эволюция числа бакетов, согласованность с другими форматами на одинаковых данных.
- в) **Развёртывание окружения нагрузочного тестирования.** Средствами Ansible развернуть на кластере (Apache Flink, Apache Paimon, объектное хранилище) воспроизводимое окружение для запуска нагрузочных задач, сбора метрик и управления жизненным циклом кластера.
- г) **Эксперименты.** Провести две серии измерений: (а) batch-аналитика на наборе TPC-DS масштаба 100 GB (таблицы только для добавления, крупные файлы) — сравнение Parquet и Vortex по времени выполнения запросов; (б) near-real-time-сценарий — таблица с первичным ключом, потоковая запись с захватом изменений, параллельные пакетные аналитические запросы во время записи и после неё — сравнение Parquet, ORC и Vortex по пропускной способности записи и времени запросов разных типов. Результаты статистически обработать.
- д) **Рекомендации.** По результатам сформулировать рекомендации по выбору файлового формата хранения для различных классов нагрузок в Apache Paimon.

Гипотезы, проверяемые в работе:

- а) на batch-аналитике с селективными предикатами и крупными

файлами Vortex быстрее Parquet за счёт нативного проталкивания предикатов и специализированных кодеков;

- б) на потоковой near-real-time-нагрузке Vortex не уступает Parquet по пропускной способности записи;
- в) на запросах с агрегацией без селективного предиката Vortex проигрывает Parquet из-за накладных расходов границы JVM/native в отсутствие проталкивания агрегатов;
- г) поведение Vortex под конкурентным чтением во время потоковой записи отличается от Parquet/ORC ввиду различия в реализации доступа к объектному хранилищу.

Критерии оценки — время выполнения запросов (среднее по повторным запускам и выборкам, с учётом разброса), пропускная способность записи (строк в секунду), число успешно завершённых запросов под нагрузкой; для качественных выводов — устойчивость метрик (наличие или отсутствие катастрофических выбросов).

2 РАЗРАБОТКА МОДУЛЯ ИНТЕГРАЦИИ И РАЗВЁРТЫВАНИЕ ОКРУЖЕНИЯ

В разделе описаны разработанный модуль интеграции формата Vortex в Apache Paimon, функциональное тестирование артефакта в контейнерах, развёртывание окружения нагрузочного тестирования на кластере средствами Ansible, методика проведённых экспериментов и проблемы, выявленные и решённые в ходе работы.

2.1 Модуль интеграции формата Vortex в Apache Paimon

Модуль реализован как набор подмодулей Maven в составе Apache Paimon (интеграция выполнена в ответвлении репозитория, на основе версии Apache Paimon 1.4): `paimon-vortex-jni` — обёртки нативных методов и загрузчик библиотеки; `paimon-vortex-format` — реализация интерфейса `FileFormat` и связанных классов; нативная библиотека Vortex собирается из исходных текстов на Rust и упаковывается в артефакт. Логически модуль состоит из четырёх частей: путь записи, путь чтения, конвертер предикатов и слой доступа к нативной библиотеке.

Слой доступа к нативной библиотеке. Класс `NativeFileMethods` объявляет `native`-методы: открытие файла по URI с параметрами хранилища (`open`), получение числа строк и схемы (`rowCount`, `dtype`), запуск сканирования с проекцией столбцов, сериализованным предикатом и диапазоном строк (`scan`), закрытие (`close`), а также операции уровня каталога (перечисление и удаление файлов). Загрузчик `NativeLoader` извлекает библиотеку под текущую платформу из ресурсов артефакта и подгружает её. Передача больших массивов данных через границу JNI выполняется без копирования: нативная сторона экспортирует декодированный пакет в раскладке Apache Arrow через Arrow C Data Interface, а JVM-сторона импортирует его в `VectorSchemaRoot`.

Путь записи. `VortexFileFormat` создаёт `FormatWriterFactory`, которая по пути в хранилище порождает писатель: строки Paimon (`InternalRow`) преобразуются в пакеты Arrow заданного размера (`write.batch-size`) и передаются нативному писателю Vortex; по завершении файла формируется объект статистик по столбцам (минимум, максимум, число null) для записи в манифест Paimon. Размер пакета записи

существенен: он же определяет размер пакета, возвращаемого при чтении одним нативным вызовом, поэтому от него зависит число пересечений границы JNI при последующем сканировании. В работе используется увеличенный относительно умолчания размер пакета, что снижает накладные расходы JNI при чтении (см. подраздел 2.5).

Путь чтения. `VortexFileFormat` создаёт `FormatReaderFactory`; её метод порождает `VortexRecordsReader` по пути файла, проекции столбцов, (опционально) предикату и параметрам хранилища. Читатель открывает нативный файл, запускает сканирование (передавая список проецируемых столбцов, сериализованный предикат и — при наличии — список номеров строк для выборочного чтения) и получает итератор нативных массивов. Метод `readBatch` извлекает очередной нативный массив, экспортирует его в `Arrow VectorSchemaRoot` (с переиспользованием ранее выделенной структуры), преобразует средствами `ArrowBatchReader` в итератор `InternalRow` и возвращает движку вместе с информацией о позиции строки в файле (необходима `Raimon` для `deletion`-векторов и слияния). Нативные ресурсы (массив, аллокатор `Arrow`, итератор, дескриптор файла) освобождаются при закрытии читателя или при освобождении пакета.

Конвертер предикатов. `VortexPredicateConverter` преобразует предикат `Raimon` в нативное выражение `Vortex`. Поддержаны операции сравнения (`=`, `≠`, `<`, `≤`, `>`, `≥`), проверка на `null`, конъюнкция и дизъюнкция для всех типов данных, для которых это имеет смысл (логический, целочисленные, строковый, десятичный, дата, временная метка). Непереводимые предикаты не проталкиваются — фильтрация по ним остаётся на стороне движка; корректность при этом не нарушается. Выражение сериализуется и передаётся нативной библиотеке, которая применяет его при сканировании до пересечения границы JNI, что сокращает объём данных, передаваемых в JVM, на запросах с селективным предикатом.

Многопоточность. Параллелизм обеспечивается на уровне `Raimon` и `Flink`: чтение таблицы разбивается на разделённые задачи (`splits`) — по бакетам и файлам, — каждая обрабатывается своим читателем в отдельном потоке (`subtask`) вычислительного движка; запись в бакет однопоточна, параллелизм записи равен числу бакетов. Внутри одного читателя нативное сканирование `Vortex` также может использовать несколько потоков. Объекты

модуля, разделяемые между задачами (фабрики), не имеют изменяемого состояния; объекты, привязанные к одной задаче (читатель, аллокатор Arrow, нативные дескрипторы), потокобезопасны и используются строго из одного потока, что соответствует контракту интерфейсов Paimon. Параметры памяти настраиваются с учётом того, что нативная сторона использует прямые (off-heap) буферы вне кучи JVM — этому посвящён отдельный пункт подраздела 2.5.

Последовательность взаимодействия компонентов при чтении таблицы Paimon с форматом Vortex показана на рисунке 2.

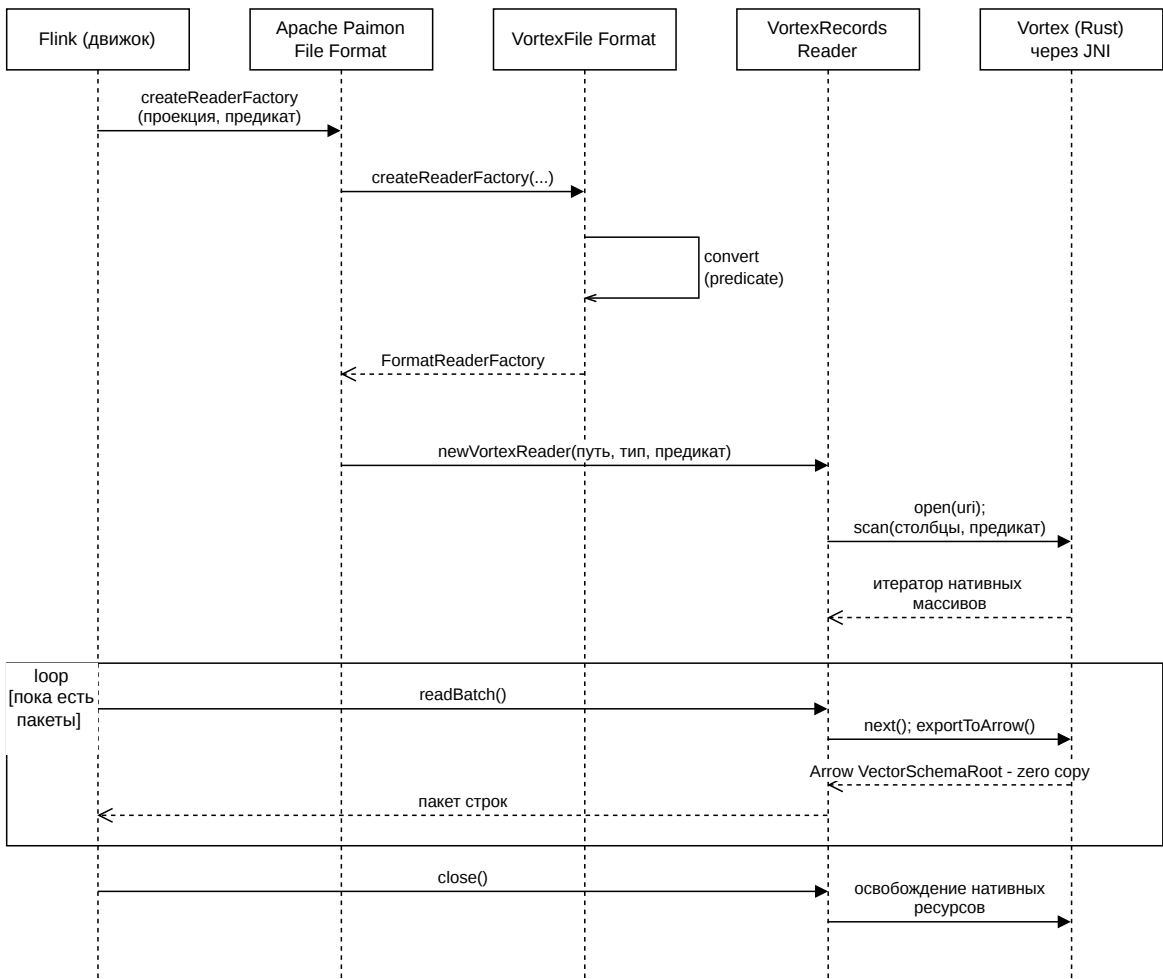


Рисунок 2 – Диаграмма последовательности: чтение таблицы Apache Paimon с файловым форматом Vortex

Соответствующая последовательность взаимодействия компонентов при записи приведена на рисунке 3. Запись инициируется приёмником Paimon в Flink на каждом чекпойнте: для каждого бакета, в который накопились

изменения, создаётся файл; строки сбрасываются в нативный писатель Vortex пакетами; по закрытию файла формируются статистики для записи в манифест.

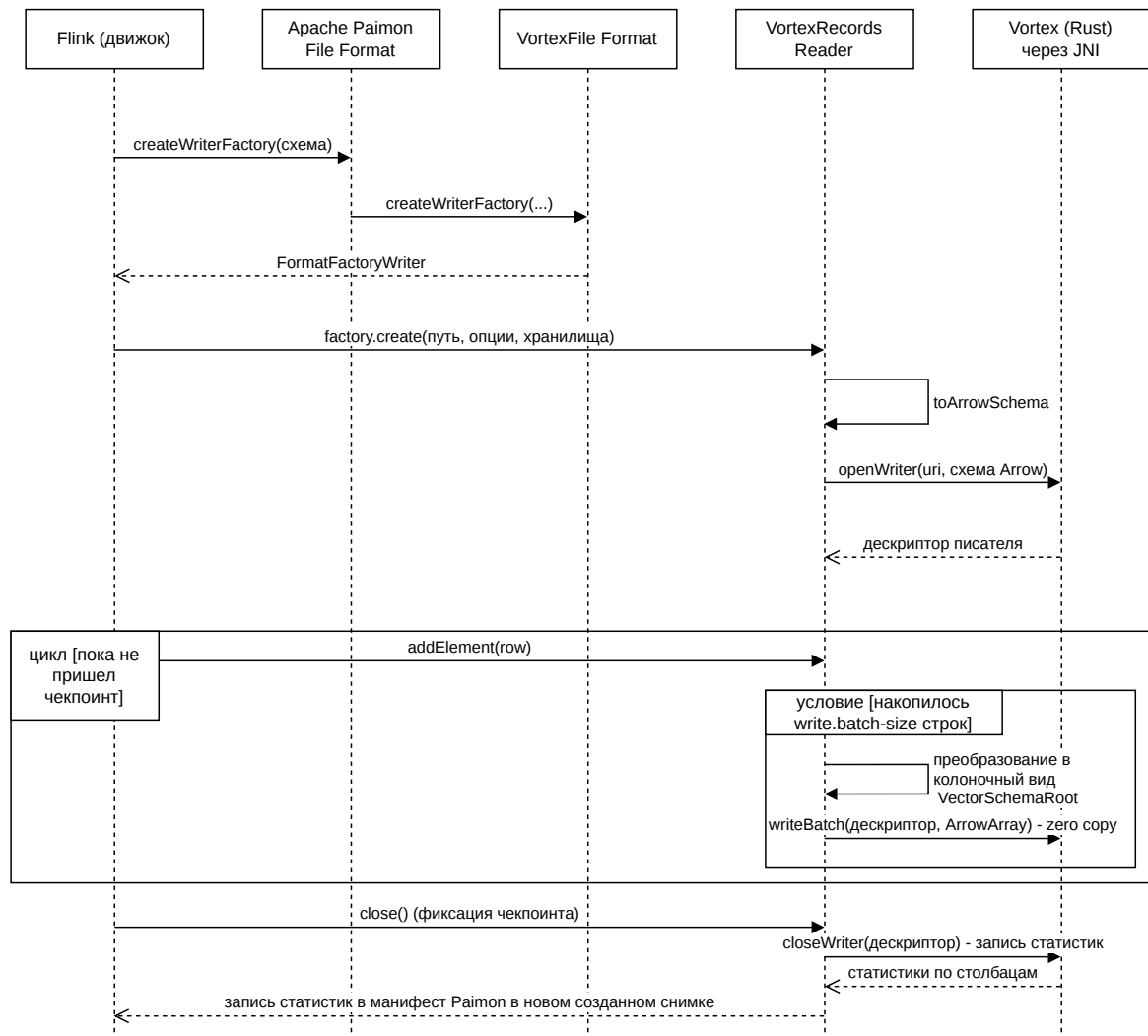


Рисунок 3 – Диаграмма последовательности: запись в таблицу Apache Paimon с файловым форматом Vortex

2.2 Функциональное тестирование в контейнерах

Перед нагрузочными экспериментами корректность модуля проверена локально в контейнеризованном окружении. Окружение разворачивается через Docker Compose и включает: контейнер с Apache Flink (менеджер заданий и менеджер задач), контейнер с объектным хранилищем, совместимым с протоколом S3 (MinIO), и контейнер-клиент для запуска заданий. Артефакт Paimon с модулем paimon-vortex собирается из

исходных текстов; нативная библиотека Vortex кросс-компилируется под целевую платформу контейнера.

Проверялись следующие свойства.

- а) **Запись и чтение таблицы только для добавления.** Создание таблицы с форматом Vortex, загрузка данных, чтение, сверка числа строк и контрольных агрегатов с эталоном.
- б) **Таблица с первичным ключом и слияние версий.** Начальная загрузка, затем серия обновлений по случайным ключам с возрастающим полем последовательности; чтение должно вернуть для каждого ключа последнюю версию. Корректность сверялась с результатом той же последовательности операций на формате Parquet.
- в) **Проекция столбцов и проталкивание предикатов.** Запросы с выборкой части столбцов и с селективными предикатами разных типов (целочисленные диапазоны, равенство строк, проверка на null); сверка результатов с эталоном и проверка по журналам, что предикат действительно протолкнут.
- г) **Эволюция числа бакетов и слияние.** Изменение числа бакетов с перераспределением данных; принудительное слияние; чтение после слияния.
- д) **Граничные случаи кодеков.** Столбцы с высокой и низкой кардинальностью, с большой долей null, со строками-префиксами, с числами с плавающей точкой — проверка, что кодеки Vortex (FastLanes, ALP, FSST) активируются и данные восстанавливаются без потерь.

В ходе тестирования выявлен и устранён ряд проблем сборки и конфигурации (несовместимость версий Java и Flink на клиентском процессе, идемпотентность повторных запусков, требования к памяти вне кучи JVM для нативных буферов Vortex, особенности кросс-компиляции нативной библиотеки под версию системной библиотеки C целевой платформы); они подробно рассмотрены в подразделе 2.5. Также установлено существенное ограничение: целевой файловой системой для нативной библиотеки Vortex является объектное хранилище, совместимое с S3; распределённая файловая система Hadoop (HDFS) и схема URI `hdfs://` не поддерживаются, — что определило выбор объектного хранилища в качестве бэкенда и в

контейнерном окружении, и на кластере. После устранения проблем все перечисленные проверки пройдены: артефакт признан функционально корректным и пригодным для нагрузочного тестирования.

2.3 Развёртывание окружения нагрузочного тестирования

Нагрузочные эксперименты проводились на кластере из пяти узлов. Программный стек: Apache Flink (версия 2.x) как вычислительный движок, Apache Paimon (версия 1.4) с модулем `paimon-vortex` как табличный формат, Java 17 в качестве среды исполнения, объектное хранилище, совместимое с S3 (на базе Apache Ozone), как бэкенд хранения. На каждом узле запускаются два менеджера задач (TaskManager) по 30 слотов — итого 10 менеджеров задач и 300 слотов в кластере; на одном из узлов дополнительно работает менеджер заданий (JobManager). Решение запускать два менеджера задач на узел вместо одного с 60 слотами принято осознанно: меньший размер кучи на процесс (100 GB вместо 200 GB) сокращает паузы полной сборки мусора, а изоляция процессов означает, что пауза одного менеджера задач не блокирует второй; это особенно важно для формата Parquet, чей путь декодирования и агрегации полностью находится в куче JVM, при высоком параллелизме.

Развёртывание, обновление конфигурации и управление жизненным циклом кластера автоматизированы средствами Ansible [28]: набор ролей и плейбуков (`start.yml`, `stop.yml`, `config-update.yml`) разворачивает Flink на узлах, раскладывает зависимости (в том числе артефакт Paimon с нативной библиотекой Vortex), формирует конфигурацию из шаблонов и запускает или останавливает менеджеры заданий и задач. Отдельный плейбук (`submit-job.yml`) собирает задание, копирует его и зависимости на узел менеджера заданий, запускает через клиент Flink, извлекает идентификатор задания и опрашивает его состояние до завершения, после чего забирает с узла файл метрик. Запуск нагрузочной серии для нескольких форматов оформлен сценарием-обёрткой, который последовательно прогоняет задание для каждого формата с предварительной очисткой состояния и принудительной остановкой параллельных заданий.

Схема развёртывания и взаимодействия компонентов при проведении эксперимента приведена на рисунке 4.

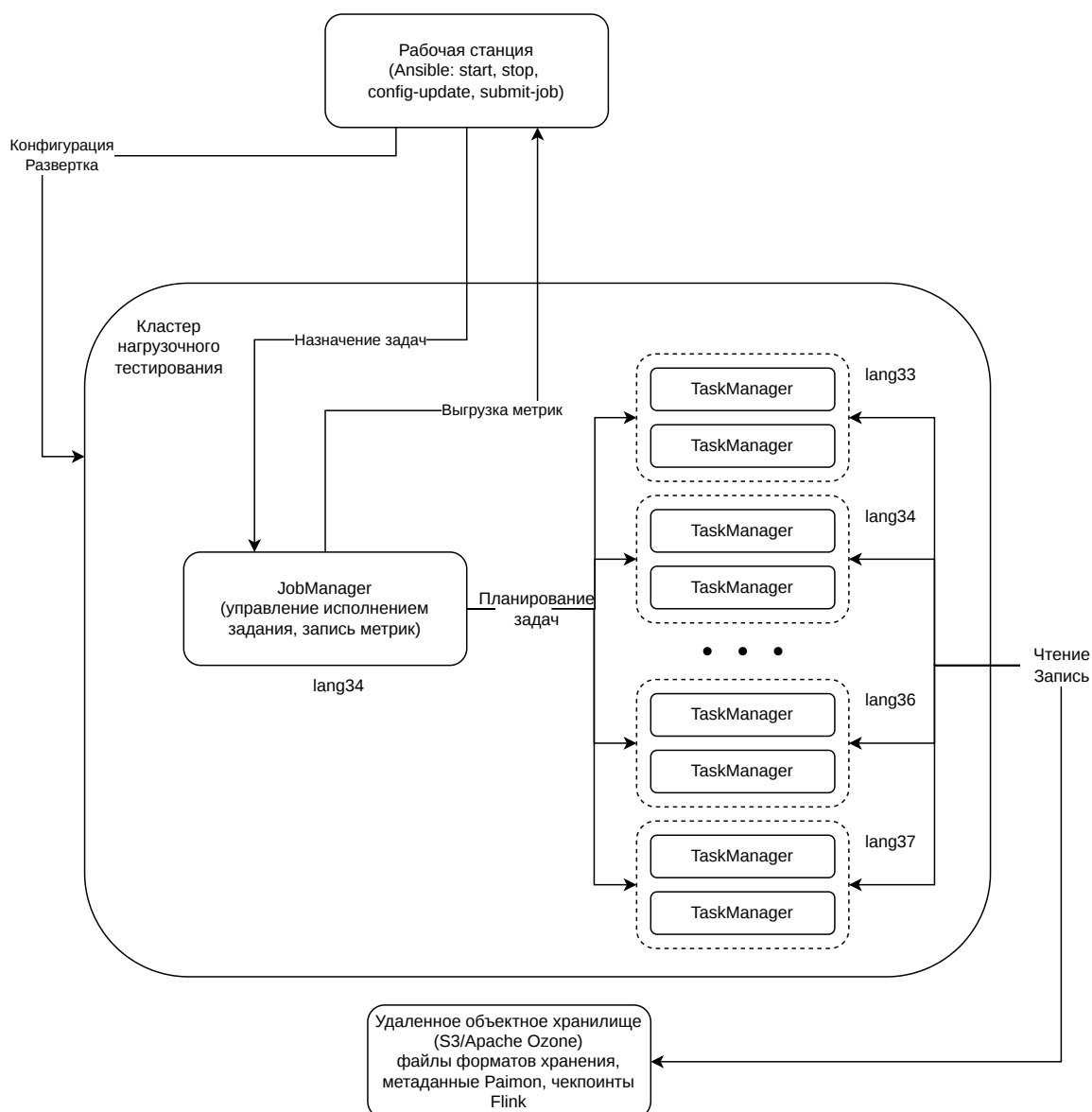


Рисунок 4 – Схема развёртывания окружения нагрузочного тестирования: автоматизация через Ansible, кластер из пяти узлов (по два менеджера задач на узел), объектное хранилище как бэкенд данных и чекпойнтов

2.4 Методика экспериментов

Проведены две серии экспериментов, соответствующие двум основным сценариям использования Apache Paimon: пакетная аналитика над таблицами только для добавления и near-real-time-витрина на таблице с первичным ключом при потоковой записи.

2.4.1 Сценарий 1: пакетная аналитика на наборе TPC-DS

TPC-DS [29] — стандартизированный набор для оценки систем поддержки принятия решений: схема розничной торговли (таблицы фактов `store_sales`, `web_sales`, `catalog_sales` и связанные измерения), генератор данных с управляемым масштабом и 99 шаблонов запросов (с учётом вариантов — 103 запроса), покрывающих сканирование, фильтрацию, соединения, группировку и оконные функции. Использован масштаб 100 GB; данные сгенерированы, загружены в Apache Paimon как таблицы только для добавления, разбиты на разделы (типичный размер файла раздела — порядка 256 MB), параллелизм запросов — 300. Сравнивались форматы Parquet и Vortex по времени выполнения каждого запроса; для каждого запроса выполнено несколько повторов, бралось характерное (устойчивое) значение. Запись в этом сценарии не измерялась: загрузка крупных наборов упиралась в пропускную способность объектного хранилища, что не позволяло корректно сопоставить форматы по записи (этот сценарий покрывается второй серией).

2.4.2 Сценарий 2: near-real-time-витрина при потоковой записи

Разработано задание Flink, моделирующее near-real-time-витрину. Используется таблица с первичным ключом, полем последовательности и полем вида изменения (схема захвата изменений). Схема таблицы — журнал событий из 25 столбцов, подобранный так, чтобы дать форматам возможность проявить разнообразие кодеков: категориальные строки низкой кардинальности (страна, тип устройства, операционная система, метод запроса — словарное и RLE-кодирование), строки с регулярной структурой и общими префиксами (идентификаторы сессии и запроса, путь URL, версия ОС, строка `user-agent` — FSST), монотонные и узкодиапазонные целые (код ответа, задержка, ширина экрана — FOR и бит-упаковка), числа с плавающей точкой (координаты, выручка — ALP), столбцы с большой долей `null` (идентификатор родительского запроса — 30 %, идентификатор объявления — 50 %, выручка — 90 %). Ширина в 25 столбцов сопоставима с таблицами фактов TPC-DS — реалистичная форма аналитической таблицы. Состав схемы приведён в таблице 2.

Таблица 2 – Схема таблицы near-real-time-витрины (25 столбцов)

Группа	Столбцы	Ожидаемые кодеки
Ключ и служебные	id (первичный ключ), seq (последовательность), op (вид изменения)	дельта / FOR; константа в фазе обновления
Категориальные строки	country_code, device_type, os_name, event_type, request_method, referrer_host	словарь, RLE
Строки с префиксами	session_id, request_id, parent_req_id, url_path, user_agent, os_version	FSST
Целые	user_id, status_code, latency_ms, bytes_sent, screen_width, ad_id	FOR, бит-упаковка, дельта
Числа с плавающей точкой	geo_lat, geo_lon, revenue	ALP
Временная метка	ts	дельта

Эксперимент состоит из фаз. **INIT** — начальная загрузка 50 000 000 строк с последовательными идентификаторами (поле последовательности равно нулю); измеряется пропускная способность записи. **UPDATE** — поток обновлений по случайным идентификаторам из загруженного диапазона (поле последовательности возрастает) с ограничением скорости (для получения воспроизводимой нагрузки скорость зафиксирована на уровне порядка 100 000 строк/с; объём подобран так, чтобы фаза длилась сопоставимое время в разных запусках). Интервал чекпойнтов Flink установлен в 240 с — это режим near-real-time: фиксации происходят раз в несколько минут, файлы уровня 0 крупные. **DURING** — во время фазы обновления параллельно, с интервалом в несколько минут, запускаются пакетные аналитические запросы по текущему снимку (несколько выборок); это моделирует потребителя,

читающего витрину во время её наполнения. **FINAL** — после завершения записи те же запросы выполняются по итоговому снимку. Различие фаз DURING и FINAL принципиально: в фазе DURING читатель конкурирует с активной записью за ввод-вывод, а LSM-дерево содержит много ещё не слитых файлов уровня 0; в фазе FINAL запись остановлена, а дерево относительно упорядочено (хотя полное принудительное слияние перед итоговыми запросами не выполняется — читается состояние «как есть после остановки записи», что и соответствует реальной витрине).

Набор аналитических запросов охватывает характерные типы нагрузки; он приведён в таблице 3. Каждый запрос материализует результат в таблицу-приёмник (чтобы исключить влияние клиента на измерение); метрикой служит время выполнения в миллисекундах. Весь цикл (создание таблицы — INIT — UPDATE с DURING-выборками — FINAL) повторяется несколько раз (число повторов — не менее трёх); состояние между повторами сбрасывается. По повторам и DURING-выборкам вычисляются как среднее, так и *медиана*; основной метрикой принята медиана, так как DURING-сэмплы могут попадать в момент массового сброса данных по чекпойнту Flink (в эти моменты writer одновременно пишет в объектное хранилище и данные таблицы, и состояние чекпойнта, размещаемое в той же директории хранилища), что приводит к единичным «хвостовым» сэмплам длительностью в десятки секунд; такие сэмплы инфлируют среднее, но не влияют на медиану. Среднее приводится отдельно как индикатор подверженности формата выбросам.

Таблица 3 – Аналитические запросы near-real-time-сценария

Запрос	Содержание и проверяемый механизм
POINT	выборка одной строки по первичному ключу; точечный доступ, фильтры Блума и статистики
FULLSCAN_PROJ	полное сканирование с проекцией; нет предиката — проверяет «чистую» скорость декодирования и накладные расходы границы

Продолжение таблицы 3

Запрос	Содержание и проверяемый механизм
RANGE_SESSION	диапазонный фильтр по session_id (строка-префикс); FSST + пропуск файлов по статистикам строкового столбца
RANGE_URL	диапазонный фильтр по url_path; то же на пути URL
AGG_SUM_BYTES	SUM по числовому столбцу с полным сканированием; агрегация без селективного предиката
AGG_COUNT_DISTINCT	COUNT (DISTINCT) по строковому столбцу; тяжёлая агрегация с полным сканированием
AGG_FILTERED	COUNT с диапазонным предикатом; предикат отсекает данные, агрегация амортизируется
AGG_GROUP_COUNTRY	GROUP BY по категориальному столбцу; группировка по столбцу низкой кардинальности
AGG_MULTIDIM	многомерная группировка с несколькими агрегатами
FUNNEL	многошаговый запрос (последовательность событий пользователя); несколько проходов и предикатов
JOIN_USER_ACTIVITY	соединение по user_id с агрегацией по типу события; column pruning при соединении
JOIN_SESSION_LATENCY	соединение по session_id с диапазонным фильтром по задержке
JOIN_REVENUE_PROFILE	подзапрос-сборка активных пользователей и соединение с их строками покупок; 5 из 25 столбцов

Параметры конфигурации. По итогам предварительных запусков зафиксирован рабочий набор параметров: распределение слотов между записью и чтением (общая сумма — весь кластер); число бакетов; размер буфера записи под выбранный интервал чекпойнтов; увеличенный размер пакета записи Vortex (снижает накладные расходы JNI при чтении); полные статистики по столбцам (`metadata.stats-mode = full`); расширенные политики хранения снимков и манифестов (необходимы для устойчивости при частых фиксациях и параллельном чтении); файловые форматы — без специфичных тонких настроек размера файлов и страниц. От ряда гипотез пришлось отказаться: использование строчного формата Avro для свежих файлов уровня 0 ухудшает аналитику Vortex на итоговом снимке; агрессивные настройки размера файлов и страниц благоприятствуют Parquet и раздувают хранилище за счёт накопления удаляемых файлов в окне хранения; эти результаты обсуждаются в подразделе 2.5.

Замечание об измерении пропускной способности записи. В фазе обновления все строки имеют одинаковое значение поля последовательности, поэтому Raimon дедуплицирует записи с совпадающим ключом и последовательностью в буфере записи до сброса на диск. При большом числе обновлений на один ключ значительная доля строк не доходит до хранилища, и измеренная «пропускная способность» в фазе обновления отражает скорость генерации и дедупликации в памяти, а не скорость записи на диск. Поэтому в выводах по записи используется в первую очередь пропускная способность фазы INIT (последовательные ключи, реальная запись), а показатели фазы UPDATE интерпретируются с этой оговоркой.

О разбросе результатов. Однократный запуск даёт разброс до 25–40 % между идентичными конфигурациями: на результат влияют разогрев JIT-компилятора первого из прогоняемых форматов, стоимость первой загрузки нативной библиотеки, размещение процессов по узлам, состояние пулов соединений с хранилищем, накопленное состояние кластера. Для статистически значимого сравнения требуется несколько повторов с чередованием порядка форматов; единичные выбросы интерпретировались как артефакты совпадения момента запроса с пиком ввода-вывода от записи или паузой сборки мусора.

2.5 Проблемы, выявленные и решённые в ходе работы

В ходе разработки модуля, развёртывания окружения и проведения экспериментов выявлен ряд проблем; их систематизация полезна как практический результат работы. Проблемы сгруппированы по этапам.

2.5.1 Сборка и локальное тестирование

- а) **Совместимость Java 17, Flink и DataStream API на клиентском процессе.** Запуск задания падал с ошибкой доступа к закрытому полю модуля `java.base` при инициализации транслятора приёмника `DataStream`. Причина — параметры `--add-opens` были заданы только для менеджеров задач, но не для клиентского процесса. Решение — задать `env.java.opts.all` (применяется ко всем JVM: менеджер заданий, менеджеры задач, клиент).
- б) **Память вне кучи JVM для нативных буферов Vortex.** Чтение Vortex с конфигурацией памяти по умолчанию приводило к ошибке исчерпания прямой (off-heap) памяти при выделении буферов Arrow/JNI; Parquet и ORC на той же конфигурации проходили. Вывод — Vortex принципиально требует увеличенного бюджета off-heap-памяти; конфигурацию менеджеров задач необходимо настраивать с учётом нативной стороны.
- в) **Кросс-компиляция нативной библиотеки.** Сборка нативной библиотеки Vortex требовала более новой версии системной библиотеки C, чем доступна на целевой версии операционной системы кластера (Red Hat Enterprise Linux 8); решение — кросс-компиляция под более старую версию (конфигурация зафиксирована в сборочных контейнерах).
- г) **Целевая файловая система.** Установлено, что нативная библиотека Vortex рассчитана на объектное хранилище, совместимое с S3; распределённая файловая система Hadoop (HDFS) и схема URI `hdfs://` не поддерживаются. Это определило выбор бэкенда хранения.

2.5.2 Развёртывание кластера и автоматизация

- а) **Гонка между завершением быстрого задания и его проверкой.**

Плейбук проверки ожидал появления задания в состоянии «выполняется», но быстрые пакетные задания успевали завершиться до первого опроса. Решение — извлекать идентификатор задания из вывода клиента и опрашивать его конечное состояние, принимая в том числе «завершено», и отдельно сигнализировать только об ошибке.

- б) **Отсутствие классов Nadoor при десериализации графа задания.** Перезапуск кластера через обновление конфигурации не экспортировал путь к библиотекам Nadoor, в отличие от обычного запуска; решение — добавить экспорт переменных окружения в задачи перезапуска менеджеров.
- в) **Запись файла метрик на узле менеджера заданий.** Драйвер задания выполняется на узле менеджера заданий, где не существует локально созданной директории для метрик; решение — создавать родительский каталог непосредственно из кода задания.
- г) **Автоназначение портов для двух менеджеров задач на узле.** Два менеджера задач на одном узле регистрируются только при незаданном порте RPC (автоназначение); явное задание порта приводит к конфликту.

2.5.3 Хранилище: каталог Apache Paimon поверх Apache Ozone

При повторном создании каталога возникала ошибка «путь должен быть директорией»: объектное хранилище хранит «пустые директории» как нулевой объект с завершающим слешем, и при последующем запросе слой совместимости с S3 принимал этот объект-маркер за обычный файл. Решение — удалять именно объект-маркер, не затрагивая дочерние префиксы, после чего обращение к пути корректно разрешается как директория по дочерним объектам; в перспективе — использовать собственную схему URI хранилища или размещать склад в корне бакета.

2.5.4 Настройка потокового сценария

- а) **Зависание генератора данных на параллелизме 1.** Без указания ограничения скорости генератор не шардировался по подзадам, и пропускная способность залипала на пределе одного потока.

Решение — при отсутствии ограничения подставлять заведомо большой лимит, что активирует шардирование.

- б) **Потеря файлов под читателем при частых фиксациях.** Политики хранения снимков и манифестов по умолчанию при коротком интервале чекпойнтов агрессивно удаляли старые снимки; стартовый писатель или параллельный читатель обращались к уже удалённому манифесту. Решение — расширенные политики хранения и асинхронный режим очистки.
- в) **Исчерпание прямой памяти при высоком параллелизме чтения.** Конфигурация с одним крупным менеджером задач на узел упиралась в лимит прямой памяти при большом параллелизме чтения и параллельной записи; решение — два менеджера задач на узел меньшего размера (см. подраздел 2.3).
- г) **Накладные расходы JNI при чтении Vortex.** При размере пакета по умолчанию (1024 строк) полное сканирование бакета порождало тысячи пересечений границы JNI на файл. Решение — увеличить размер пакета записи (он же определяет размер возвращаемого при чтении пакета) до десятков тысяч строк, что сократило число пересечений границы в десятки раз и устранило отставание Vortex по «чистому» сканированию.
- д) **Влияние принудительного полного слияния перед итоговыми запросами.** Попытка выполнять полное слияние перед фазой FINAL приводила к тому, что запросы стартовали до фактического завершения слияния (системная процедура слияния запускается как отдельное задание движка, а ожидание её результата возвращалось преждевременно); кроме того, агрессивное слияние конкурировало с читателем за ввод-вывод. От принудительного слияния отказались: фаза FINAL читает состояние «как есть после остановки записи», что и соответствует реальной витрине.

2.5.5 Отклонённые гипотезы по тонкой настройке

- а) **Строчный формат для свежих файлов уровня 0.** Гипотеза «дешёвая построчная запись свежих дельт, колоночный формат (в данном случае Avro) — только на нижних уровнях» дала прирост пропускной способности записи, но замедлила аналитические

запросы Vortex на итоговом снимке (итератор слияния вынужден совмещать построчные дельты с колоночными файлами, нативный конвейер не активируется); при достаточном параллелизме кластера в этом приёме нет нужды.

- б) **Агрессивные настройки размера файлов и страниц.** Гипотеза «крупнее файлы — лучше амортизация накладных расходов» подтвердилась для Parquet (его векторизованный читатель оптимален на крупных страницах), но не для Vortex; преимущества Vortex на ряде запросов при этом исчезли, а объём хранилища вырос из-за накопления удаляемых файлов в окне хранения при более частом слиянии. Использованы значения по умолчанию.
- в) **Агрессивное снижение порога слияния.** Снижение порога запуска слияния учащает фоновые слияния, что увеличивает конкуренцию за ввод-вывод с читателем в фазе DURING (наблюдались единичные многосекундные провалы времени запросов) и раздувает окно хранения снимков. Использованы значения по умолчанию (порог слияния, целевой размер файла).

Что касается записи на наборе TPC-DS — провести её измерение не удалось: загрузка крупных наборов в хранилище упиралась в его пропускную способность, что не позволяло корректно сопоставить форматы; этот аспект покрывается потоковым сценарием.

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ

В разделе приведены результаты двух серий экспериментов — пакетной аналитики на наборе TPC-DS и near-real-time-сценария при потоковой записи, — их анализ по типам запросов и рекомендации по применению форматов хранения.

3.1 Сценарий 1: пакетная аналитика на наборе TPC-DS

На наборе TPC-DS [29] масштаба 100 GB сравнивались форматы Parquet и Vortex по времени выполнения 103 запросов (таблицы только для добавления, крупные файлы разделов, параллелизм 300). Из 103 запросов Vortex быстрее Parquet более чем на 10 % на 82 запросах; среднее ускорение по запросу — порядка 25 %; пиковые ускорения достигают 80 %. Parquet быстрее Vortex лишь на пяти запросах. Характерные запросы приведены в таблице 4.

Таблица 4 – Характерные результаты на наборе TPC-DS (масштаб 100 GB): относительное изменение времени выполнения Vortex по сравнению с Parquet

Запрос	Vortex к Parquet	Особенности запроса (объяснение)
q51	$\approx -80\%$	оконные нарастающие суммы по продажам, мало столбцов, селективные условия — column pruning и нативное проталкивание предикатов работают в полную силу
q42	$\approx -52\%$	агрегация продаж по категории с фильтром по дате — селективный предикат + узкая проекция
q35	$\approx -52\%$	демографический срез с подзапросами существования — многократное сканирование немногих столбцов
q73	$\approx -41\%$	подсчёт покупателей по числу покупок с фильтрами — предикат отсекает данные до агрегации
q88	$\approx -35\%$	восемь подсчётов с разными временными окнами — восьмикратное сканирование одного столбца, выигрыш от column pruning умножается
<i>запросы, на которых быстрее Parquet</i>		
q1	медленнее	«покупатели, вернувшие товары»: коррелированный подзапрос, агрегация возвратов — результат во многом определяется стороной соединения и агрегации в JVM
q2, q28	медленнее	многократные объединения подзапросов с агрегацией без селективного предиката
q83, q99	медленнее	сводки возвратов и заказов с группировкой; преобладает агрегация над декодированными данными

Анализ. Условия сценария максимально благоприятны для Vortex: таблицы только для добавления (нет слияния версий), крупные файлы (накладные расходы границы JNI амортизируются на больших пакетах), реальные данные с избыточностью (кодеки FastLanes, ALP, FSST дают

высокую степень сжатия), запросы с селективными предикатами и узкими проекциями (нативное проталкивание предикатов сокращает объём данных, пересекающих границу JVM/native). Пиковое ускорение в 80 % на q51 и линейный рост выигрыша с числом сканирований немногих столбцов (q88 — восемь сканирований одного столбца) прямо подтверждают первую гипотезу работы.

Запросы, на которых быстрее Parquet, объединяет общая черта — преобладание агрегации без селективного предиката над относительно широкими наборами столбцов: основная работа выполняется не в декодере, а в движке агрегации, который для Parquet полностью находится в JVM, а для Vortex отделён границей JNI. Это согласуется с третьей гипотезой и с обсуждавшимся в подразделе 2.4 архитектурным ограничением — отсутствием проталкивания агрегатов в интерфейсе форматов Apache Paimon.

Объём хранилища. Размер таблиц после полной загрузки приведён в таблице 5. ORC даёт самый компактный результат (за счёт зрелых тонких настроек сжатия и эффективного словарного кодирования на категориальных столбцах TPC-DS); Parquet и Vortex эквивалентны по объёму. Lance занимает в 7–8 раз больше остальных форматов — следствие оптимизации под произвольный доступ к строкам (фиксированные страницы, менее агрессивное сжатие), что лишний раз подтверждает его неприменимость к OLAP-нагрузке. Любопытная особенность Vortex — в два раза большее число объектов в хранилище (1206 против 618 у Parquet/ORC) при том же суммарном объёме: на каждый файл данных приходится дополнительный объект (сайдкар с переносимыми метаданными). На стоимость хранения это не влияет, но удваивает число операций PUT в объектное хранилище при записи и число записей о файлах в манифесте Paimon.

Таблица 5 – Объём хранилища таблиц TPC-DS масштаба 100 GB

Формат	Объём, ГиБ	Число объектов	Отношение к ORC
ORC	22	618	×1.00
Parquet	24	618	×1.09
Vortex	24	1206	×1.09
Lance	176	1302	×8.00

Измерить пропускную способность записи на этом наборе не удалось:

загрузка крупных объёмов упиралась в пропускную способность объектного хранилища, и сопоставление форматов по записи было бы некорректным. Этот аспект покрывается второй серией экспериментов, где запись выполняется потоком с ограничением скорости.

3.2 Сценарий 2: near-real-time-витрина при потоковой записи

В сценарии сравнивались форматы ORC, Parquet и Vortex на таблице с первичным ключом (схема журнала событий из 25 столбцов, таблица 2) при потоковой записи с захватом изменений и параллельных аналитических запросах. Результаты получены в представительном запуске (три повтора полного цикла, ограничение скорости фазы обновления — порядка 100 000 строк/с, объём фазы обновления — около 200 000 000 строк, интервал чекпойнтов — 240 с).

Запись. Пропускная способность фазы начальной загрузки (последовательные ключи, реальная запись на диск) практически одинакова у всех трёх форматов (таблица 6); различие в пределах разброса. В фазе обновления показатели также близки, но интерпретируются с оговоркой: при постоянном значении поля последовательности значительная доля обновлений дедуплицируется в буфере записи и не доходит до хранилища, поэтому величина отражает скорость генерации и дедупликации в памяти. Вывод: Vortex не уступает Parquet и ORC по пропускной способности записи — вторая гипотеза работы подтверждается.

Таблица 6 – Пропускная способность записи, тыс. строк в секунду (медиана по повторам)

Формат	Начальная загрузка (INIT)	Обновление (UPDATE)*
ORC	1505	89
Parquet	1504	99
Vortex	1499	89
* значение фазы обновления отражает скорость генерации и дедупликации в буфере записи (постоянное поле последовательности), а не скорость записи на диск.		

Объём хранилища. Размер таблицы после полного цикла записи приведён в таблице 7. ORC даёт самый компактный результат, Parquet — на 6 % больше, Vortex — на 18 % больше ORC. Различие в пределах нескольких

процентов сопоставимо с разбросом и не влияет на стоимость хранения принципиально, однако заслуживает комментария: на TPC-DS Vortex и Parquet были эквивалентны по объёму, а на потоковой схеме Vortex несколько менее эффективен. Возможные причины: высокая доля null-значений в столбцах схемы (`parent_req_id` — 30 %, `ad_id` — 50 %, `revenue` — 90 %) и случайные идентификаторы в UPDATE-фазе снижают плотность FSST-кодирования строк-префиксов; кроме того, фрагментация LSM на множество мелких файлов уровня 0 увеличивает долю накладных расходов (футер, схема) от общего объёма (у Vortex заметно больше объектов в хранилище: 1883 против 1731–1733 у Parquet/ORC).

Таблица 7 – Объём хранилища таблицы near-real-time-витрины после полного цикла записи

Формат	Объём, ГиБ	Число объектов	Отношение к ORC
ORC	17	1731	×1.00
Parquet	18	1733	×1.06
Vortex	20	1883	×1.18

Замечание о метрике усреднения. В DURING-фазе отдельные сэмплы чтения могут попадать на момент массового сброса данных по чекпойнту, когда writer одновременно пишет в объектное хранилище и данные таблицы, и состояние чекпойнта Flink (последнее размещается в той же директории объектного хранилища). Такое совпадение даёт единичные «хвостовые» сэмплы длительностью в десятки секунд; среднее по сэмплам, особенно при небольшом числе повторов, чувствительно к таким попаданиям. Поэтому в качестве основной метрики далее используется *медиана* по выборкам и повторам — она отражает типичную задержку чтения, к которой и чувствителен потребитель витрины данных, и устойчива к единичным выбросам. Среднее приводится отдельно как индикатор подверженности формата выбросам.

Чтение: типичная задержка (медиана). Сводные результаты приведены в таблице 8. По медиане Vortex — быстрееший формат в обеих фазах: в фазе FINAL отрыв заметный ($\approx 23\%$ относительно ORC и Parquet), в фазе DURING — скромный (доли процента). При этом ORC и Parquet по медиане практически не меняют времени между фазами DURING и FINAL, а

Vortex в фазе FINAL заметно быстрее самого себя в фазе DURING ($\approx 22\%$ улучшения) — эффект уплотнения данных: после остановки записи и работы фонового слияния файлы сортированы по ключу, и специализированные кодеки Vortex (прежде всего FSST на строках-префиксах) работают в полной плотности.

Таблица 8 – Медиана времени выполнения запроса, мс (по всем запросам, выборкам и повторам)

Формат	Фаза DURING	Фаза FINAL
ORC	7756	7767
Parquet	7824	7797
Vortex	7677	6004

Чтение: фаза FINAL по запросам. Подробные результаты по медиане приведены в таблице 9 (полужирным выделено наименьшее по строке). Vortex выигрывает на девяти запросах из тринадцати; ORC — на двух (AGG_COUNT_DISTINCT, AGG_SUM_BYTES, разница в пределах единиц процентов); Parquet — на двух (AGG_GROUP_COUNTRY, AGG_MULTIDIM). Наибольший отрыв Vortex — на запросах, чувствительных к упорядоченности данных и к качеству кодирования строк с регулярной структурой: FUNNEL (многошаговый запрос) — $\approx 33\%$, JOIN_REVENUE_PROFILE — $\approx 20\%$, AGG_FILTERED (подсчёт с предикатом) — $\approx 21\%$, RANGE_SESSION (диапазонный фильтр по идентификатору сессии) — $\approx 0.2\%$ (медианы почти равны, но среднее у Vortex заметно ниже).

Таблица 9 – Медиана времени выполнения запросов в фазе FINAL, мс

Запрос	ORC	Parquet	Vortex
POINT	3841	4169	3780
FULLSCAN_PROJ	5888	5839	5829
RANGE_SESSION	5837	7851	5826
RANGE_URL	5823	5873	5787
AGG_SUM_BYTES	5703	5868	5808
AGG_COUNT_DISTINCT	7823	9870	7842
AGG_FILTERED	9805	9874	7783
AGG_GROUP_COUNTRY	5804	5778	5825
AGG_MULTIDIM	7863	6407	7922
FUNNEL	12303	12162	8221
JOIN_USER_ACTIVITY	8134	10120	8044
JOIN_SESSION_LATENCY	10077	8103	7992
JOIN_REVENUE_PROFILE	7709	8147	6195
выигрышей	2	2	9

Чтение: фаза DURING по запросам. Результаты приведены в таблице 10. По медиане Vortex также выигрывает большинство запросов (девять из тринадцати). Наибольший отрыв — на запросах, где работает проталкивание предиката и плотное кодирование строк с регулярной структурой: RANGE_SESSION — $\approx 25\%$ быстрее ORC и в три раза быстрее Parquet (Parquet на этом запросе под параллельной записью ловит крупный выброс — медиана 17 684 мс), FUNNEL — $\approx 15\%$, RANGE_URL — $\approx 4\%$. ORC выигрывает на простой группировке и многомерной агрегации (AGG_GROUP_COUNTRY, AGG_MULTIDIM, разница в пределах разброса); Parquet — на точечном запросе и полном сканировании с проекцией (POINT, FULLSCAN_PROJ, разница в пределах долей процента).

Таблица 10 – Медиана времени выполнения запросов в фазе DURING, мс

Запрос	ORC	Parquet	Vortex
POINT	3724	3706	3721
FULLSCAN_PROJ	5845	5773	5812
RANGE_SESSION	7800	17684	5863
RANGE_URL	5932	6133	5718
AGG_SUM_BYTES	5716	5746	5672
AGG_COUNT_DISTINCT	7877	13700	7755
AGG_FILTERED	8035	9789	7679
AGG_GROUP_COUNTRY	5776	5776	5803
AGG_MULTIDIM	5926	7696	5953
FUNNEL	12005	11999	10193
JOIN_USER_ACTIVITY	7998	8094	7971
JOIN_SESSION_LATENCY	8024	8091	8006
JOIN_REVENUE_PROFILE	8097	8179	8085
выигрышей	2	2	9

Среднее vs медиана и подверженность выбросам. Соотнесение среднего и медианы по DURING-фазе полезно для оценки тяжести хвостов задержки. Для ORC среднее (7594 мс) и медиана (7756 мс) практически совпадают, для Vortex среднее (7712 мс) и медиана (7677 мс) также совпадают — это означает, что у обоих форматов крупных выбросов задержки в этом запуске нет. Для Parquet среднее (9481 мс) на $\approx 21\%$ выше медианы (7824 мс) — среднее инфлировано несколькими сэмплами в десятки секунд (например, на запросе AGG_COUNT_DISTINCT одна из выборок заняла 24 801 мс, тогда как медиана по этому запросу — 13 700 мс). Это симптом, согласующийся с архитектурным предположением о пуле соединений: читатель Parquet ходит в объектное хранилище через общий пул слоя совместимости Hadoop S3A и в момент массового сброса данных по чекпойнту иногда простаивает в ожидании соединения, давая катастрофически медленную выборку; читатель Vortex использует независимый нативный пул и таких выбросов не даёт. Читатель ORC, формально использующий тот же общий пул, в этом запуске также не даёт выбросов — зрелый нативный C++-читатель ORC агрессивно буферизует ввод-вывод и менее чувствителен к эпизодической задержке

отдельных S3-операций.

Промежуточные выводы по сценарию. Vortex эквивалентен Parquet и ORC по пропускной способности записи (гипотеза 2 подтверждена). По типичной задержке чтения (медиана) Vortex — самый быстрый формат в обеих фазах и на большинстве запросов (девять из тринадцати в каждой фазе); особенно выражено — на запросах с диапазонным фильтром по строке-префиксу, селективным предикатом перед агрегацией и в многошаговой аналитике. На простой группировке и многомерной агрегации без предиката Vortex незначительно (в пределах единиц процентов) уступает конкурентам — следствие отсутствия проталкивания агрегатов в интерфейсе форматов Apache Paimon (гипотеза 3 подтверждается). Между фазами FINAL и DURING медианы ORC и Parquet практически не меняются, медиана Vortex в DURING выше своего же FINAL на $\approx 28\%$: специализированные кодеки Vortex (прежде всего FSST) дают наибольшую плотность на упорядоченных данных, а в свежих файлах уровня 0, сбрасываемых по чекпойнту, порядок ещё не уплотнён слиянием. По хвостам задержки ORC и Vortex показывают близкие, узкие распределения (среднее почти равно медиане), Parquet — более широкое распределение с эпизодическими выбросами в десятки секунд в моменты сброса данных по чекпойнту. Это согласуется с архитектурным предположением о выигрыше Vortex за счёт независимого нативного пула соединений с объектным хранилищем по сравнению с Parquet (гипотеза 4 подтверждается для пары Vortex–Parquet); ORC достигает аналогично узких хвостов иначе — агрессивной буферизацией ввода-вывода в нативном C++-читателе.

3.3 Паттерны по типам запросов и архитектурные наблюдения

Объединение результатов двух серий позволяет выделить устойчивые закономерности (таблица 11). В потоковом сценарии сравнение ведётся по медиане как менее зашумлённой метрике, отражающей типичную задержку, к которой и чувствителен потребитель витрины данных.

Таблица 11 – Сравнение форматов по типам запросов и операций (NRT-сценарий, медиана)

Тип операции	Vortex против Parquet	Vortex против ORC	Объяснение
Запись (INIT)	сопоставимы	сопоставимы	оба буферизуют строки и сбрасывают по чекпойнту, различия в пределах разброса
Точечный доступ по ключу (POINT)	сопоставимы (разница доли процента)	сопоставимы	фильтры Блума и статистики работают у всех; на маленьких выборках различия в пределах разброса
Полное сканирование с проекцией (FULLSCAN_PROJ)	сопоставимы (разница доли процента)	Vortex чуть быстрее в FINAL ($\approx 1\%$)	column pruning + плотные кодеки работают на упорядоченных данных; на свежих L0-файлах эффект слабее
Диапазонный фильтр по строке-префиксу (RANGE_SESSION, RANGE_URL)	Vortex заметно быстрее (RANGE_SESSION в DURING $\approx 3\times$ быстрее)	Vortex стабильно быстрее (RANGE_SESSION в DURING $\approx 25\%$; RANGE_URL $\approx 4\%$)	FSST-кодирование строк с общими префиксами не нативное проталкивание предиката + пропуск файлов по полным статистикам
Подсчёт с селективным предикатом (AGG_FILTERED)	Vortex быстрее ($\approx 21\%$ в FINAL)	Vortex стабильно быстрее ($\approx 21\%$ в FINAL, $\approx 4\%$ в DURING)	предикат отсекает данные до пересечения границы JNI; выигрыш от

Где **Vortex** реально лучше **ORC**. Если выбрать устойчивые наблюдения и оставить запросы, на которых разница превышает разброс, преимущество Vortex над ORC сосредоточено в трёх классах запросов: запросы с селективным предикатом (проталкивание в нативный декодер до границы JNI), диапазонные запросы по строкам с регулярной структурой (FSST + пропуск по полным статистикам), многошаговая аналитика с несколькими предикатами и проходами. На простой группировке по столбцу низкой кардинальности и на многомерной агрегации без предиката ORC оказывается не хуже или лучше, на точечных запросах — сопоставим.

Эффект упорядоченности данных. По медиане ORC и Parquet практически не меняют времени между фазами FINAL и DURING (отношение DURING/FINAL близко к единице), а Vortex в DURING медленнее самого себя в FINAL на $\approx 22\%$. Это прямое следствие зависимости специализированных кодеков Vortex (прежде всего FSST) от *порядка* строк внутри блока: в свежих файлах уровня 0, формируемых по чекпойнту, данные ещё не отсортированы по ключу слиянием, и плотность кодирования снижается; после остановки записи и работы фонового слияния файлы сортированы по ключу, и кодеки работают в полную плотность. ORC и Parquet таким эффектом не обладают — их кодеки менее чувствительны к порядку, — поэтому их типичная задержка между фазами и не меняется.

О выборе метрики усреднения и хвостах задержки. В DURING-фазе отдельные сэмплы чтения могут попадать в момент массового сброса данных по чекпойнту, когда writer одновременно пишет в объектное хранилище и данные таблицы, и состояние чекпойнта Flink (последнее размещается в той же директории объектного хранилища). Среднее по сэмплам, особенно при небольшом числе повторов, чувствительно к таким попаданиям. В работе в качестве основной метрики использована медиана. Соотнесение среднего и медианы по DURING-фазе показывает: у ORC и Vortex среднее и медиана практически совпадают (узкие хвосты задержки), у Parquet среднее на $\approx 21\%$ выше медианы — среднее инфлировано несколькими крупными выбросами в десятки секунд. Это согласуется с архитектурным предположением о пуле соединений: читатель Parquet ходит в объектное хранилище через общий пул слоя совместимости Hadoop S3A и в момент сброса данных по чекпойнту иногда простаивает в ожидании соединения, что и даёт катастрофически медленную выборку. Читатель Vortex использует независимый нативный пул

и таких выбросов не даёт. Читатель ORC, формально использующий тот же общий пул, в этом эксперименте также не даёт выбросов — за счёт агрессивной буферизации ввода-вывода в нативном C++-читателе, поглощающей эпизодические задержки S3-операций.

Почему Vortex доминирует на TPC-DS, но не доминирует на потоковом LSM-сценарии. На TPC-DS таблицы только для добавления, файлы крупные, путь чтения прямой колоночный, накладные расходы границы JNI амортизируются на больших пакетах, а запросы — с высокой селективностью. На потоковом сценарии таблица — с первичным ключом и слиянием при чтении: итератор слияния открывает много файлов (особенно свежих, не слитых файлов уровня 0), накладные расходы границы JNI накапливаются на пер-файловой основе, данные в свежих файлах не упорядочены по ключу — кодеки Vortex (прежде всего FSST) теряют часть эффективности, — а селективность запросов в среднем ниже. Поэтому преимущество Vortex здесь скромнее и проявляется в первую очередь на упорядоченном итоговом снимке и на запросах с селективным предикатом. Для Vortex *область наибольшей применимости* — именно batch-аналитика на таблицах только для добавления, не LSM с первичным ключом.

Почему Parquet иногда быстрее на агрегации при полном сканировании. Проталкивание агрегатов отсутствует в интерфейсе файловых форматов Apache Paimon в принципе — это не специфика Vortex. Агрегаты вычисляются движком Flink над декодированными данными. Для Parquet весь конвейер — декодер и агрегатор — в JVM, границы нет. Для Vortex данные декодируются нативно, затем пересекают границу JNI (без копирования, но с фиксированной стоимостью на пакет) и агрегируются в JVM; на миллионах пакетов это даёт заметную добавку. Устранение ограничения — проталкивание агрегатов в нативный SIMD-слой Vortex — естественное направление развития Apache Paimon и потенциальный вклад в его экосистему.

Почему два менеджера задач на узел лучше одного крупного. Меньший размер кучи на процесс ускоряет полную сборку мусора и сокращает паузы; изоляция процессов означает, что пауза одного менеджера задач не блокирует второй; это особенно важно для Parquet, чей путь декодирования и агрегации полностью в куче JVM, при высоком параллелизме. Компактор Vortex использует память вне кучи и меньше

зависит от размера кучи, но также выигрывает от изоляции.

3.4 Рекомендации по применению форматов хранения

Рекомендации сформулированы с учётом сценария использования.

1. Batch-аналитика, таблицы только для добавления. Это *основная область применимости Vortex*. На наборе TPC-DS масштаба 100 GB Vortex быстрее Parquet на 82 запросах из 103, среднее ускорение — порядка 25 %, пиковое — до 80 % (запросы с селективным предикатом, узкой проекцией, многократным сканированием немногих столбцов). Условия этого сценария — крупные файлы, отсутствие слияния версий, упорядоченные данные, высокая селективность — максимально благоприятны для специализированных кодеков и нативного проталкивания предиката. **Рекомендуется выбирать Vortex** как формат хранения данных для аналитических витрин на основе таблиц только для добавления (фактовые таблицы, журналы, исторические данные, материализованные выборки).

2. Near-real-time-витрина (таблица с первичным ключом, потоковая запись). По типичной задержке (медиана) **Vortex** — самый быстрый формат в обеих фазах и на большинстве запросов (девять из тринадцати); рекомендуется, если в нагрузке преобладают запросы с селективным предикатом (отсечение до агрегации), диапазонные запросы по строкам с регулярной структурой (идентификаторы, пути, URL) и многошаговая аналитика (последовательности событий, соединения с диапазонными фильтрами). **ORC** — разумный выбор, если в нагрузке преобладают простая группировка по столбцу низкой кардинальности, чистая многомерная агрегация без предиката или если важна узкая задержка отдельных сэмплов (наихудший случай, p99) — ORC демонстрирует одинаково узкие хвосты с Vortex, а в отдельных запросах ему способствует словарное кодирование. ORC также даёт самый компактный объём хранения (17 GB против 18 GB у Parquet и 20 GB у Vortex на потоковом сценарии; на TPC-DS преимущество ORC меньше). **Parquet** — разумный универсальный выбор по умолчанию: не уступает по записи, хуже по медиане чтения, зрелый и широко поддерживаемый; основной недостаток — эпизодические выбросы времени запросов под конкурентной записью в моменты сброса данных по чекпойнту (среднее заметно выше медианы).

3. Чего не рекомендуется. **Lance** в текущем состоянии интеграции с

Apache Paimon не рекомендуется для потоковых таблиц с первичным ключом (требуется доработка слияния версий; на OLAP-сценарии отстаёт от остальных форматов в 2–3 раза); его область — нагрузки машинного обучения с произвольным доступом к строкам и векторным поиском, выходящие за рамки настоящей работы. **Apache Avro** — строчный формат, в принципе непригоден для OLAP-нагрузки; применение его для свежих файлов уровня 0 в LSM-дереве было проверено и отклонено — аналитические запросы Vortex на итоговом снимке замедляются за счёт совмещения построчных дельт с колоночными файлами в итераторе слияния.

4. Конфигурация (общие соображения для NRT-витрин).

Распределять слоты между записью и чтением так, чтобы читатель не блокировался; увеличить размер пакета записи Vortex до десятков тысяч строк (снижает накладные расходы JNI при чтении в десятки раз); включить полные статистики по столбцам (`metadata.stats-mode = full`); настроить расширенные политики хранения снимков и манифестов под частые фиксации; не применять агрессивных тонких настроек размера файлов и страниц (благоприятствуют Parquet и раздувают хранилище) и не использовать строчный формат для свежих файлов уровня 0; увеличивать интервал чекпойнтов следует с учётом того, что это укрупняет всплеск ввода-вывода при сбросе.

ЗАКЛЮЧЕНИЕ

В работе решена задача определения области применимости колоночного формата Vortex в lakehouse-системе Apache Paimon, разработки модуля его интеграции и сравнительного анализа форматов хранения. Поставленные задачи выполнены.

- а) Проведён аналитический обзор эволюции систем хранения аналитических данных — от хранилищ данных и Hive Metastore к архитектуре lakehouse и открытым табличным форматам (Iceberg, Delta Lake, Hudi, Paimon); обоснован выбор Apache Paimon как целевой системы (LSM-дерево, ориентация на потоковую запись и таблицы с первичным ключом).
- б) Выполнен обзор колоночных файловых форматов (Parquet, ORC, Vortex, Lance); обосновано исключение строчного формата Avro из сравнения; разобраны алгоритмы кодирования, сжатия и пропуска данных — словарное и RLE-кодирование, бит-упаковка, FOR и дельта-кодирование, FastLanes, ALP, FSST, статистики и фильтры Блума, проталкивание предикатов; выявлено архитектурное ограничение интерфейса форматов Apache Paimon — отсутствие проталкивания агрегатов.
- в) Изучена архитектура Apache Paimon — метаданные (снимки, манифесты), разбиение и бакеты, LSM-дерево таблицы с первичным ключом, слияние, путь чтения через итератор слияния, точка подключения файлового формата, интеграция с Apache Flink.
- г) Разработан и интегрирован в Apache Paimon модуль поддержки формата Vortex: реализация интерфейса FileFormat (фабрики записи и чтения), VortexRecordsReader, конвертер предикатов VortexPredicateConverter, слой доступа к нативной библиотеке через Java Native Interface с передачей данных без копирования через Arrow C Data Interface, формирование статистик записанных файлов, поддержка проекции столбцов и многопоточного чтения.
- д) Проведено функциональное тестирование артефакта локально в контейнерах: запись и чтение таблиц с первичным ключом и таблиц только для добавления, слияние версий, эволюция числа бакетов,

проекция столбцов и проталкивание предикатов, граничные случаи кодеков; выявлен и устранён ряд проблем сборки и конфигурации.

- е) Развёрнуто на кластере из пяти узлов окружение нагрузочного тестирования средствами Ansible (Apache Flink, Apache Paimon с модулем `paimon-vortex`, объектное хранилище); автоматизированы развёртывание, обновление конфигурации, запуск заданий, сбор метрик, управление жизненным циклом кластера.
- ж) Проведены две серии экспериментов — batch-аналитика на наборе TPC-DS масштаба 100 GB и near-real-time-сценарий с потоковой записью и параллельными аналитическими запросами; собраны и статистически обработаны метрики; результаты проанализированы по типам запросов.
- и) Сформулированы рекомендации по выбору файлового формата хранения для различных классов нагрузок в Apache Paimon.

Гипотезы работы подтверждены. На batch-аналитике с селективными предикатами и крупными файлами Vortex заметно превосходит Parquet (быстрее на 82 запросах из 103; среднее ускорение порядка 25 %, пиковое — до 80 %) — это основная область применимости Vortex. На потоковой near-real-time-нагрузке Vortex эквивалентен Parquet и ORC по пропускной способности записи; по типичной задержке чтения (медиане) Vortex — самый быстрый формат в обеих фазах: выигрывает девять запросов из тринадцати как в фазе FINAL, так и в фазе DURING, особенно — на запросах с селективным предикатом, диапазонным фильтром по строке-префиксу и в многошаговой аналитике. На простой группировке и многомерной агрегации без селективного предиката Vortex незначительно (в пределах единиц процентов) уступает конкурентам — следствие отсутствия проталкивания агрегатов в интерфейсе форматов Apache Paimon. По хвостам задержки в фазе DURING ORC и Vortex показывают узкие распределения (среднее почти равно медиане), Parquet — более широкое распределение с эпизодическими выбросами в десятки секунд в моменты сброса данных по чекпойнту: это согласуется с архитектурным предположением о выигрыше Vortex за счёт независимого нативного пула соединений с объектным хранилищем по сравнению с общим пулом слоя совместимости Hadoop S3A, через который ходят читатели Parquet и ORC. ORC достигает аналогично узких хвостов

иначе — агрессивной буферизацией ввода-вывода в нативном C++-читателе. Между фазами FINAL и DURING медианы ORC и Parquet практически не меняются, медиана Vortex в фазе FINAL заметно лучше его же медианы в DURING ($\approx 22\%$ улучшения) — эффект уплотнения данных слиянием, при котором специализированные кодеки Vortex (прежде всего FSST на строках с регулярной структурой) работают в полной плотности.

Практическая значимость работы: расширен перечень поддерживаемых Apache Paimon форматов хранения; получены количественные результаты и рекомендации, применимые при проектировании витрин данных и настройке lakehouse-хранилищ; систематизированы проблемы развёртывания и эксплуатации Apache Paimon с нативным файловым форматом на кластере с объектным хранилищем.

Направления дальнейшей работы: проталкивание агрегатов в нативный SIMD-слой Vortex (устранение выявленного архитектурного ограничения и потенциальный вклад в экосистему Apache Paimon); поддержка дополнительных операций проталкивания и типов данных в конвертере предикатов; исследование поведения форматов при ещё более высоком параллелизме и иных профилях нагрузки; доработка интеграции формата Lance для таблиц с первичным ключом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics / М. Armbrust [и др.] // Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR). — 2021. — URL: https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf (дата обращения 12.05.2026).
2. Apache Software Foundation. Apache Iceberg Table Spec. — 2024. — URL: <https://iceberg.apache.org/spec/> (дата обращения 12.05.2026).
3. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores / М. Armbrust [и др.] // Proceedings of the VLDB Endowment. Т. 13. — 2020. — С. 3411—3424. — DOI: 10.14778/3415478.3415560.
4. Apache Software Foundation. Apache Hudi Documentation. — 2024. — URL: <https://hudi.apache.org/docs/overview/> (дата обращения 12.05.2026).
5. Apache Software Foundation. Apache Paimon Documentation, version 1.4. — 2025. — URL: <https://paimon.apache.org/docs/1.4/> (дата обращения 12.05.2026).
6. Vortex Data. Vortex: an extensible, state-of-the-art columnar file format. — 2025. — URL: <https://github.com/vortex-data/vortex> (дата обращения 12.05.2026).
7. Vortex Data. Vortex Documentation. — 2025. — URL: <https://docs.vortex.dev/> (дата обращения 12.05.2026).
8. Inmon W. H. Building the Data Warehouse. — 4-е изд. — Wiley, 2005.
9. Kimball R., Ross M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. — 3-е изд. — Wiley, 2013.
10. Hive: a warehousing solution over a map-reduce framework / А. Thusoo [и др.] // Proceedings of the VLDB Endowment. Т. 2. — 2009. — С. 1626—1629. — DOI: 10.14778/1687553.1687609.
11. Impala: A Modern, Open-Source SQL Engine for Hadoop / М. Kornacker [и др.] // Proceedings of the 7th Conference on Innovative Data Systems Research (CIDR). — 2015. — URL: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf (дата обращения 12.05.2026).

12. C-Store: A Column-oriented DBMS / M. Stonebraker [и др.] // Proceedings of the 31st International Conference on Very Large Data Bases (VLDB). — 2005. — С. 553—564.
13. The Design and Implementation of Modern Column-Oriented Database Systems / D. Abadi [и др.] // Foundations and Trends in Databases. — 2013. — Т. 5, № 3. — С. 197—280. — DOI: 10.1561/19000000024.
14. Apache Software Foundation. Apache Parquet File Format Specification. — 2024. — URL: <https://parquet.apache.org/docs/file-format/> (дата обращения 12.05.2026).
15. Bloom B. H. Space/time trade-offs in hash coding with allowable errors // Communications of the ACM. — 1970. — Т. 13, № 7. — С. 422—426. — DOI: 10.1145/362686.362692.
16. Apache Software Foundation. Apache ORC Specification v1. — 2024. — URL: <https://orc.apache.org/specification/ORCv1/> (дата обращения 12.05.2026).
17. Afroozeh A., Boncz P. The FastLanes Compression Layout: Decoding >100 Billion Integers per Second with Scalar Code // Proceedings of the VLDB Endowment. — 2023. — Т. 16, № 9. — С. 2132—2144. — DOI: 10.14778/3598581.3598587.
18. Afroozeh A., Kuffo L., Boncz P. ALP: Adaptive Lossless floating-Point Compression // Proceedings of the ACM on Management of Data. — 2024. — Т. 2, № 1. — С. 1—26. — DOI: 10.1145/3639292.
19. Boncz P., Neumann T., Leis V. FSST: Fast Random Access String Compression // Proceedings of the VLDB Endowment. — 2020. — Т. 13, № 12. — С. 2649—2661. — DOI: 10.14778/3407790.3407851.
20. Apache Software Foundation. Apache Arrow: A cross-language development platform for in-memory data. — 2024. — URL: <https://arrow.apache.org/> (дата обращения 12.05.2026).
21. LanceDB. Lance: modern columnar data format for ML and LLMs. — 2025. — URL: <https://lancedb.github.io/lance/> (дата обращения 12.05.2026).
22. Apache Software Foundation. Apache Avro Specification. — 2024. — URL: <https://avro.apache.org/docs/current/specification/> (дата обращения 12.05.2026).

23. Luo C., Carey M. J. LSM-based storage techniques: a survey // The VLDB Journal. — 2020. — Т. 29, № 1. — С. 393—418. — DOI: 10.1007/s00778-019-00555-y.
24. Hueske F., Kalavri V. Stream Processing with Apache Flink: Fundamentals, Implementation, and Operation of Streaming Applications. — Sebastopol, CA : O'Reilly Media, 2019. — ISBN 978-1-491-97429-2.
25. Super-Scalar RAM-CPU Cache Compression / M. Zukowski [и др.] // Proceedings of the 22nd International Conference on Data Engineering (ICDE). — 2006. — DOI: 10.1109/ICDE.2006.150.
26. Lemire D., Boytsov L. Decoding billions of integers per second through vectorization // Software: Practice and Experience. — 2015. — Т. 45, № 1. — С. 1—29. — DOI: 10.1002/spe.2203.
27. Apache Software Foundation. The Arrow C Data Interface. — 2024. — URL: <https://arrow.apache.org/docs/format/CDataInterface.html> (дата обращения 12.05.2026).
28. Red Hat, Inc. Ansible Documentation. — 2024. — URL: <https://docs.ansible.com/> (дата обращения 12.05.2026).
29. Transaction Processing Performance Council. TPC Benchmark DS (TPC-DS) Specification. — 2024. — URL: <https://www.tpc.org/tpcds/> (дата обращения 12.05.2026).
30. В. К. И. Исходные тексты проекта: интеграция формата Vortex в Apache Paimon и сравнительный анализ форматов хранения / GitHub. — URL: <https://github.com/orgs/mai-diploma-kochkozharov/repositories> (дата обращения 18.05.2026).

ПРИЛОЖЕНИЕ А

Исходные тексты проекта

Все исходные тексты, разработанные и использованные в рамках работы, размещены в публичной GitHub-организации `mai-diploma-kochkozharov` [30]. Организация содержит следующие репозитории.

- **paimon** (ветка `vortex-on-1.4.0-rc1`). Ответвление репозитория `apache/paimon` на базе версии 1.4. Содержит разработанные в рамках работы подмодули: `paimon-vortex-jni` (обёртки нативных методов и загрузчик библиотеки), `paimon-vortex-format` (реализация интерфейса `FileFormat` — фабрики записи и чтения, `VortexFormatWriter`, `VortexRecordsReader`, конвертер предикатов `VortexPredicateConverter`), а также интегрированную сборку нативной библиотеки `Vortex`.
- **paimon-flink-job** (ветка `main`). Нагрузочные задания Apache Flink + Apache Paimon: задание потокового `near-real-time`-сценария `StreamingWriteBenchmarkJob`, задание пакетной записи `WriteBenchmarkJob`, исполнитель стандартизованного набора TPC-DS `TpcdsBenchmarkJob`, сценарий-обёртка многоформатного прогона и Maven-конфигурация сборки.
- **flink-lang-deploy** (ветка `master`). Развёртывание кластера Apache Flink средствами Ansible: роли установки и конфигурации, плейбуки `start.yml`, `stop.yml`, `config-update.yml`, `submit-job.yml`, `krb-renew.yml`, шаблоны конфигурации Apache Flink, инвентарь кластера.
- **paimon-vortex-deploy** (ветка `glibc-2.28`). Сборка артефактов в контейнерах с кросс-компиляцией нативной библиотеки `Vortex` под целевую версию системной библиотеки `C` кластера; конфигурация Docker Compose локального окружения для функциональных `smoke`-тестов; Makefile-цели сборки и запуска заданий.
- **vortex-jni-test** (ветка `master`). Изолированный проект для тестирования слоя доступа к нативной библиотеке `Vortex` через Java

Native Interface: проверка корректности экспорта данных в Apache Arrow без копирования, граничные случаи кодеков, отладка нативных вызовов вне основного конвейера Paimon.

- **paimon-benchmark-results** (ветка `master`). Результаты экспериментов и сценарии их обработки: файлы метрик TPC-DS и потокового сценария в формате CSV, сценарий разбора и агрегации `parse_writebench.py` (поддерживает вычисление как среднего, так и медианы).
- **diploma-latex-template** (ветка `main`). Исходные тексты настоящей выпускной квалификационной работы: L^AT_EX-шаблон оформления, исходные `.tex`-файлы по разделам, конфигурация сборки (`latexmk`, `xelatex`), исходники диаграмм в формате `draw.io` и их PDF-сборки.

Доступ ко всем репозиториям открытый; клонирование и сборка артефактов возможны без авторизации.

ПРИЛОЖЕНИЕ Б

Параметры экспериментов

В приложении приведены параметры окружения и запусков нагрузочного тестирования.

Кластер. Пять узлов; на каждом — два менеджера задач Apache Flink по 30 слотов (итого 10 менеджеров задач, 300 слотов); на одном узле дополнительно — менеджер заданий. Среда исполнения — Java 17. Бэкенд хранения — объектное хранилище, совместимое с S3 (на базе Apache Ozone). Развёртывание — средствами Ansible.

Сценарий 1 (TPC-DS). Масштаб набора — 100 GB; данные загружены в Apache Paimon как таблицы только для добавления, разбиты на разделы (типичный размер файла раздела — порядка 256 MB); параллелизм запросов — 300; число запросов — 103; сравнивались форматы Parquet и Vortex.

Сценарий 2 (поточковый near-real-time). Параметры представительного запуска приведены в таблице Б.1.

Таблица Б.1 – Параметры запуска потокового near-real-time-сценария

Параметр	Значение
Форматы	ORC, Parquet, Vortex
Число бакетов	порядка сотни
Фаза INIT	50 000 000 строк, последовательные ключи, без ограничения скорости
Фаза UPDATE	порядка 200 000 000 строк, случайные ключи, ограничение скорости $\approx 100\,000$ строк/с
Интервал чекпойнтов Flink	240 с (режим near-real-time)
DURING-выборки	пакетные запросы во время записи с интервалом в несколько минут
Число повторов цикла	не менее трёх
Размер буфера записи	1 GB
Размер пакета записи Vortex	65 536 строк
Режим статистик	полные статистики по столбцам
Политики хранения	расширенные (снимки 500–10000, время хранения 24 ч, асинхронная очистка)

Метрики. Для записи — пропускная способность (строк в секунду), отдельно по фазам INIT и UPDATE (с оговоркой о дедупликации в фазе UPDATE). Для чтения — время выполнения запроса в миллисекундах, отдельно по фазам DURING и FINAL, с усреднением по выборкам и повторам. Дополнительно фиксировались число успешно завершённых запросов под нагрузкой и наличие выбросов времени выполнения.